

**LEVERAGING LARGE LANGUAGE MODELS FOR
ANOMALY DETECTION OF CYBER ADVANCED
PERSISTENT THREATS**

by

WALEED KHAN MOHAMMED

1221302664

Session 2025/2026

The project report is prepared for
Faculty of Artificial Intelligence and Engineering
Multimedia University
in partial fulfilment of
Bachelor of Engineering (Hons.) Electronics
majoring in Computer

**FACULTY OF ARTIFICIAL INTELLIGENCE AND ENGINEERING
MULTIMEDIA UNIVERSITY**

July 2026

© **2026 Universiti Telekom Sdn. Bhd. ALL RIGHTS RESERVED.**

Copyright of this report belongs to Universiti Telekom Sdn. Bhd. as qualified by Regulation 7.2 (c) of the Multimedia University Intellectual Property and Commercialisation Policy. No part of this publication may be reproduced, stored in or introduced into a retrieval system, or transmitted in any form or by any means (electronic, mechanical, photocopying, recording, or otherwise), or for any purpose, without the express written permission of Universiti Telekom Sdn. Bhd. Due acknowledgement shall always be made of the use of any material contained in, or derived from, this report.

DECLARATION

I hereby declare that this work has been done by myself and no portion of the work contained in this report has been submitted in support of any application for any other degree or qualification of this or any other university or institute of learning.

I also declare that pursuant to the provisions of the Copyright Act 1987, I have not engaged in any unauthorised act of copying or reproducing or attempt to copy/reproduce or cause to copy/reproduce or permit the copying/reproducing or the sharing and/or downloading of any copyrighted material or an attempt to do so whether by use of the University's facilities or outside networks/facilities whether in hard copy or soft copy format, of any material protected under the provisions of sections 3 and 7 of the Act whether for payment or otherwise save as specifically provided for therein. This shall include, but not be limited to, any lecture notes, course packs, thesis, textbooks, exam questions, and any works of authorship fixed in any tangible medium of expression, whether provided by the University or otherwise.

I hereby further declare that in the event of any infringement of the provisions of the Act, whether knowingly or unknowingly, the University shall not be liable for the same in any manner whatsoever and undertakes to indemnify and keep indemnified the University against all such claims and actions.

Signature: _____

Name: **WALEED KHAN MOHAMMED**

Student ID: 1221302664

Date: July 2026

ACKNOWLEDGMENT

I would like to express my deepest gratitude to the Almighty for granting me the strength, patience, and wisdom to complete this Final Year Project successfully.

I extend my sincere appreciation to my supervisor, Ir. Prof. Dr. Hezerul Bin Abdul Karim, for his invaluable guidance, continued support, and insightful feedback throughout this project. His expertise, encouragement, and constructive suggestions played a crucial role in shaping the direction and quality of this research. I am also deeply thankful to my co-supervisor, Dr. Nouar Al Dahoul, for her helpful advice, technical insights, and consistent support during the development of this work.

I am profoundly grateful to my parents for their unwavering love, encouragement, and sacrifices. Their constant motivation and belief in me have been a great source of strength and inspiration throughout my academic journey.

I would also like to thank my friends and peers for their support, cooperation, and encouragement during the course of this project. Their discussions, suggestions, and moral support contributed significantly to the successful completion of this work.

Finally, I would like to express my appreciation to everyone who has directly or indirectly contributed to the completion of this Final Year Project.

ABSTRACT

Cybersecurity operations heavily rely on signature-based or rule-based systems to detect malicious activity. However, these traditional methods struggle to detect Advanced Persistent Threats (APTs) that dynamically alter their behavior to evade fixed rules. Large Language Models (LLMs) offer a flexible alternative by learning deep semantic patterns in data rather than matching hardcoded strings. Despite their capability, deep learning frameworks act as black boxes, making them difficult to interpret. When an alert triggers, security analysts cannot easily determine why a model flagged the event, leading to alert fatigue and a lack of trust in live Security Operations Center (SOC) deployments.

This thesis addresses this gap by presenting a fully unsupervised anomaly detection framework that integrates high-dimensional semantic detection with a two-stage Explainable AI (XAI) pipeline. The core detection engine maps structured system log telemetry into text embeddings, which are then processed by autoencoder architectures to identify structural anomalies without relying on labeled attack data during training.

To determine the optimal architecture, this study conducts an extensive benchmarking evaluation encompassing 16 pre-trained LLM embedding models and 11 unique autoencoder topologies. These 176 combinations were rigorously tested across five DARPA TC dataset performers and five distinct process contexts, yielding a total of 4,400 individual experimental runs. Due to the extreme class imbalance inherent in system telemetry, Precision-Recall Area Under the Curve (PR-AUC) was utilized as the primary evaluation metric to ensure operational reliability.

The experimental results demonstrate that smaller, bidirectional encoder models (such as `gtr-t5-large` and `ModernBERT-large`) consistently outperform massive, instruction-tuned generative decoder models when processing template-generated system logs. The optimal configuration reached a peak PR-AUC of 0.9684 under the network flow context (`ProcessNetflow`).

For the explainability component, a token-level pipeline combining a TF-IDF representation with SHAP (SHapley Additive exPlanations) successfully mapped high anomaly scores back to specific adversarial syntax, such as external IP address octets and unusual executable path fragments. Furthermore, an automated multi-stage ablation study

validated the causal impact of these tokens, showing up to an 87.5% reduction in reconstruction error when the SHAP-identified threat features were systematically removed. This work proves that robust unsupervised detection and precise, causal explainability can coexist, providing a transparent foundation for next-generation intrusion detection systems.

Keywords: Advanced Persistent Threats (APTs), Anomaly Detection, Large Language Models (LLMs), Explainable AI (XAI), SHAP, Autoencoders, Cybersecurity.

TABLE OF CONTENTS

DECLARATION	1
ACKNOWLEDGMENT	2
ABSTRACT	3
TABLE OF CONTENTS	6
LIST OF TABLES	7
LIST OF FIGURES	9
LIST OF ABBREVIATIONS	10
1 INTRODUCTION	12
1.1 Overview	12
1.2 Problem Statement	14
1.3 Research Objectives	15
1.4 Project Scope	15
1.5 Significance of Study	15
1.6 Report Outline	16
2 LITERATURE REVIEW	17
2.1 Introduction	17
2.2 Cyber Advanced Persistent Threats (APTs)	17
2.3 Anomaly Detection in Cybersecurity	18
2.4 Large Language Models for Security Applications	18
2.5 Text Embeddings for Cybersecurity Log Analysis	20
2.6 Autoencoder Architectures for Anomaly Detection	20
2.7 Transformer-Based Approaches for APT Detection	22
2.8 Explainability in APT Detection	23
2.9 Research Gaps	24
3 RESEARCH METHODOLOGY	29
3.1 Introduction	29
3.2 Dataset Acquisition, Standardization, and Anti-Leakage Protocol . . .	29

3.3	Embedding Model Taxonomy and Benchmarking	32
3.3.1	Group A: The “Encoder” Family (BERT-Based)	32
3.3.2	Group B: The “Decoder” Family (LLM-Based)	33
3.3.3	Groups C, D, and E: “Large”, “Angle”, and “Static” Architectures	33
3.4	Deep Learning Architecture: Autoencoder Topology Search	34
3.5	The End-to-End Anomaly Detection Pipeline	39
3.6	The Dual-Pipeline XAI Attribution Framework	40
3.6.1	Pipeline 1: Token-Level Explainability (TF-IDF and SHAP) .	40
3.6.2	Pipeline 2: LLM Latent Space Ablation Study	40
3.7	Evaluation Metrics and Experimental Setup	41
4	RESULTS AND DISCUSSION	43
4.1	Experimental Overview	43
4.2	ProcessAll Context: Best Combinations Per Performer	43
4.3	Cross-Context Performance Analysis	46
4.4	Best Overall Configuration Per Performer	48
4.5	Explainable AI (XAI) Pipeline Results	50
4.5.1	SHAP Token Attribution Results	50
4.5.2	Latent Space Ablation Study Results	54
4.6	Summary of Key Findings	54
5	CONCLUSION AND FUTURE WORK	57
5.1	Overview of the Research	57
5.2	Summary of Key Findings and Contributions	57
5.3	Limitations of the Study	58
5.4	Future Work	59
5.5	Concluding Remarks	60
	REFERENCES	67
	LIST OF PUBLICATIONS	68

LIST OF TABLES

2.1	Summary of Literature Related to APT Detection, LLMs, Autoencoders, and Explainability	25
3.1	Experimental datasets of DARPA’s TC program used in our study. A dataset entry (columns 3 to 7) is described by the number of rows (processes) / the number of columns (attributes). For instance, with ProcessAll (PA) obtained from the second scenario of Linux, the dataset comprises 279,369 rows and 457 attributes, including 100 APT attacks (0.036%).	30
3.2	Taxonomy of LLM embedding models evaluated in this study, organized by architectural group.	34
3.3	Autoencoder topologies evaluated in the architecture search, with their regularization strategies and determinism properties.	38
3.4	Autoencoder architectures and hyperparameters used during the architecture search.	39
4.1	Best embedding and autoencoder combination per performer under the ProcessAll context, ranked by PR-AUC.	44
4.2	Best PR-AUC per process context and performer across all 176 embedding and autoencoder combinations. Bold values mark the best-performing context per performer.	46
4.3	Best overall configuration per performer across all contexts, embedding models, and autoencoder architectures with full evaluation metrics. . .	48
4.4	Automated multi-stage ablation study results. The Original rows show the full log sentence and baseline error. Removed rows show which tokens were removed and the new error score. A drop in error confirms causal attribution.	55

LIST OF FIGURES

1.1	Stages of an APT Attack	12
1.2	Growth in APT-Related Cyber Incidents Globally [1]	13
1.3	High-Level Overview of the Proposed LLM-Based APT Detection Framework	14
3.1	Overall System Architecture	30
3.2	Improved anti-leakage 80/20 dataset split protocol. The model is trained exclusively on benign records, while all anomalous records are reserved for testing. This ensures unbiased evaluation and prevents data leakage.	31
3.3	Two-dimensional t -SNE projection comparison of representative raw embedding groups. Malicious logs are marked in red while normal activities are marked in blue, demonstrating the initial cluster density before neural autoencoder filtering.	35
3.4	Enhanced visualization of autoencoder reconstruction behaviour for benign and APT logs. Benign samples align with the learned normal distribution and reconstruct with minimal error. APT samples fall outside this distribution, causing poor reconstruction and elevated mean squared error (MSE), which is used as the anomaly score.	36
3.5	Enhanced automated multi-stage ablation workflow with improved alignment and flow structure. Threat-indicative tokens identified by TF-IDF SHAP are progressively removed and re-evaluated. The resulting reduction in reconstruction error quantitatively reveals which tokens most strongly drive the anomaly signal.	41
4.1	Confusion matrices for all performers under the ProcessAll context. The matrices show classification performance across normal and anomalous log events for each dataset.	45
4.2	PR-AUC heatmaps for all five DARPA TC performers under the ProcessAll context. Rows represent embedding models and columns represent autoencoder architectures. Darker shading indicates higher PR-AUC. .	47
4.3	Confusion matrices for each performer under their respective best overall configuration across all contexts.	49
4.4	SHAP token attribution plots (Part 1).	51
4.5	SHAP token attribution plots (Part 2).	52

4.6	SHAP beeswarm plots for all DARPA TC performers. Features with larger positive SHAP values consistently increase anomaly scores, revealing the tokens that drive model decisions across the test set. . . .	53
-----	---	----

LIST OF ABBREVIATIONS

ADAPT	Audit Data for APT Analysis
AE	Autoencoder
AI	Artificial Intelligence
ALBERT	A Lite BERT
API	Application Programming Interface
APT	Advanced Persistent Threat
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BERT	Bidirectional Encoder Representations from Transformers
BSD	Berkeley Software Distribution
C2	Command and Control
CSV	Comma-Separated Values
CUDA	Compute Unified Device Architecture
DAE	Denoising Autoencoder
DARPA	Defense Advanced Research Projects Agency
DARPA TC	DARPA Transparent Computing
DistilBERT	Distilled BERT
FN	False Negative
FP	False Positive
FPR	False Positive Rate
GMVAE	Gaussian Mixture Variational Autoencoder
GNN	Graph Neural Network
GPT	Generative Pre-trained Transformer
GRU	Gated Recurrent Unit
GPU	Graphics Processing Unit
GUID	Globally Unique Identifier
IDS	Intrusion Detection System
KL	Kullback-Leibler
KDE	Kernel Density Estimation
L1	L1 Regularization (Lasso)
L2	L2 Normalization
LIME	Local Interpretable Model-Agnostic Explanations
LLM	Large Language Model
LotL	Living off the Land

LSTM	Long Short-Term Memory
LSTMAE	Long Short-Term Memory Autoencoder
MLP	Multilayer Perceptron
MiniLM	Mini Language Model
MSE	Mean Squared Error
NLP	Natural Language Processing
OCSVM	One-Class Support Vector Machine
OOV	Out of Vocabulary
PCA	Principal Component Analysis
PDF	Probability Density Function
PR AUC	Precision-Recall Area Under the Curve
RAG	Retrieval-Augmented Generation
ReLU	Rectified Linear Unit
RoBERTa	Robustly Optimized BERT Pretraining Approach
SHAP	SHapley Additive exPlanations
SOC	Security Operations Center
SVDD	Support Vector Data Description
TF-IDF	Term Frequency-Inverse Document Frequency
TN	True Negative
TP	True Positive
TPR	True Positive Rate
VAE	Variational Autoencoder
VRAM	Video Random Access Memory
WMI	Windows Management Instrumentation
XAI	Explainable Artificial Intelligence

CHAPTER 1 INTRODUCTION

1.1 Overview

Cyber threats have grown more dangerous over time, but few are as serious as Advanced Persistent Threats (APTs). Most cyberattacks are quick and opportunistic. APTs are the opposite. They are slow, deliberate, and targeted. Attackers quietly infiltrate a network and stay hidden for months, sometimes years, collecting data or waiting for the right moment to act. Their goals typically include espionage, data theft, or disrupting critical infrastructure. These attacks are usually carried out by well-funded groups, including nation-state actors and organized criminal organizations [2].

What makes APTs so hard to catch is how carefully they operate. Instead of triggering alarms, attackers mimic normal user and system behavior to blend in. They often use tools that are already installed on the target machine, a strategy called “living off the land,” which makes their activity nearly impossible to distinguish from legitimate operations [3]. Figure 1.1 shows the typical stages of an APT attack, from initial reconnaissance all the way through to data exfiltration.

The impact of APTs has been significant and growing. Figure 1.2 shows the rise in reported APT incidents over recent years, reflecting both their increasing frequency and their expanding reach across sectors including government, healthcare, finance, and energy [1,4].

Traditional security tools such as signature-based Intrusion Detection Systems (IDS)



Figure 1.1: Stages of an APT Attack

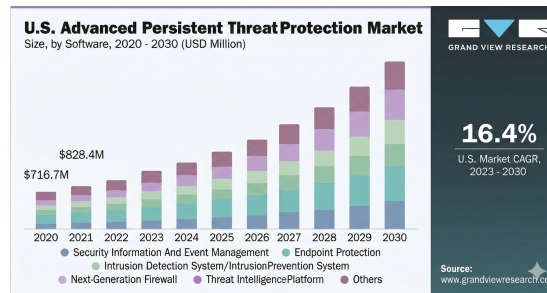


Figure 1.2: Growth in APT-Related Cyber Incidents Globally [1]

and antivirus software are poorly suited to catching APTs. They work by matching activity against known attack patterns. If the attack is new or well-disguised, these tools miss it entirely. This fundamental limitation has pushed researchers toward behavior-based approaches that can detect anomalies even without prior knowledge of the specific attack.

One practical direction is using the logs that systems generate automatically. Every process execution, file access, network connection, and login leaves a TRACE. The challenge is making sense of these large, noisy log streams and identifying the small fraction of events that signal something malicious. This is where Large Language Models (LLMs) become useful. LLMs are models pre-trained on large amounts of text that convert any text input into numerical representations called embeddings. When system log events are written as natural language sentences and passed through an LLM, the resulting vectors capture not just what happened, but the surrounding context. This makes it much easier to separate normal behavior from suspicious behavior.

This project builds on that idea. The framework takes audit logs from the ADAPT repository, converts them into sentences, encodes them using pre-trained LLM embedding models, and applies unsupervised anomaly detection to identify APT activity. Figure 1.3 provides a high-level overview of the proposed framework.

The project runs in two phases. Phase 1 used the `all-mpnet-base-v2` embedding model with a Vanilla Autoencoder, benchmarked against Isolation Forest, One-Class SVM (OCSVM), and PCA. The Autoencoder achieved the best AUC-ROC of 0.95, and these results were published at the Multimedia Engineering Conference 2026. Phase 2 significantly expands the scope by testing 17 embedding models across five architectural families and 11 Autoencoder topologies. SHAP explainability is added to make detection decisions interpretable. These results were submitted to a Q1 high IF journal.

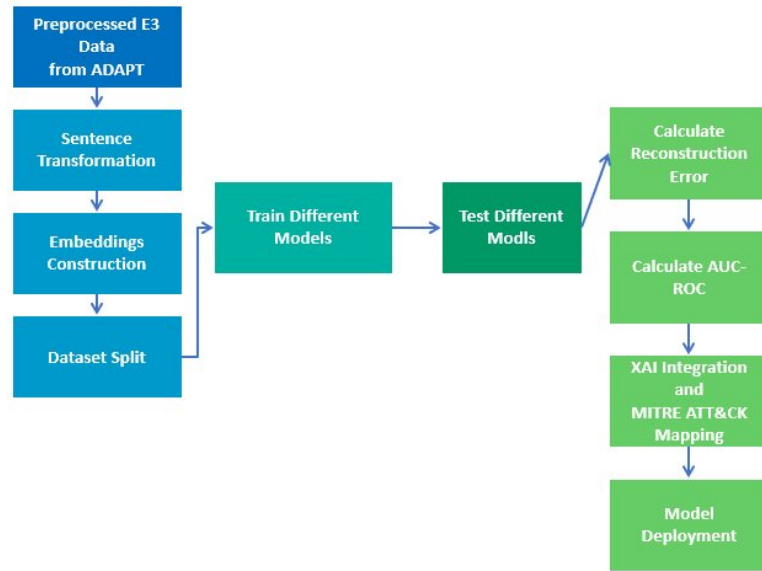


Figure 1.3: High-Level Overview of the Proposed LLM-Based APT Detection Framework

1.2 Problem Statement

APT detection remains one of the hardest problems in cybersecurity for several concrete reasons.

First, APTs are designed to be invisible. They unfold across multiple stages, each blending into normal system activity. No single event looks suspicious on its own. It is only when events are viewed in sequence and in context that the attack becomes visible. Most detection systems are not built to reason at that level [4].

Second, signature-based tools fail against APTs by design. They rely on catalogued, known attack patterns. APT actors specifically adapt their techniques to avoid matching any known signature, regularly changing their tools, tactics, and procedures (TTPs). This makes signature-based systems structurally unsuitable as the primary defense against APTs [5].

Third, most machine learning approaches to log analysis treat log entries as isolated data points, discarding the sequential context that reveals the bigger picture. A log entry showing `powershell.exe` executed is ambiguous on its own. Placed in the context of unusual network connections and a newly created scheduled task, it tells a very different story. LLM embeddings are well-positioned to capture this kind of context, but there has been limited systematic work comparing which embedding models and autoencoder architectures actually perform best on APT log data.

Finally, even accurate detection is not enough on its own. Analysts need to understand

why something was flagged before they can act on it. A model that produces an anomaly score with no explanation creates uncertainty rather than clarity. This lack of interpretability is a practical barrier to deploying these systems in real security operations.

1.3 Research Objectives

The objectives of this project are:

1. To develop a novel anomaly detection framework utilizing Large Language Models (LLMs) for identifying Advanced Persistent Threats (APTs).
2. To evaluate the accuracy, scalability, and effectiveness of LLM-based detection mechanisms compared to traditional cybersecurity methods.
3. To create a robust interpretability component within the detection system that explains the rationale behind threat identification decisions to cybersecurity analysts.

1.4 Project Scope

This project focuses on log-based APT detection using the ADAPT repository dataset. It does not cover real-time detection or live deployment in a Security Operations Center (SOC), though the framework is designed with that extension in mind.

All models are trained in a fully unsupervised manner using only benign samples, which reflects the real-world constraint that labelled APT data is scarce. The embedding models evaluated are pre-trained models sourced from HuggingFace and the OpenAI API. No models are trained from scratch on cybersecurity data within this project.

Explainability through SHAP is applied to the best-performing configurations from Phase 2 as a post-hoc analysis.

1.5 Significance of Study

This project makes three main contributions. Technically, it provides one of the most comprehensive comparisons of LLM embedding models and autoencoder architectures applied specifically to APT detection, giving the research community clear empirical guidance on what works in this domain.

Practically, integrating SHAP transforms the system from a black-box anomaly scorer

into something an analyst can actually work with. Knowing which features drove a detection decision makes alert triage faster and more reliable. Instead of just knowing that something is anomalous, an analyst can immediately see what type of threat behavior it resembles, which significantly speeds up investigation and response.

1.6 Report Outline

Chapter 2 reviews existing literature on APTs, anomaly detection, LLM embeddings, autoencoder architectures, and explainability in security systems, and identifies the gaps this project addresses.

Chapter 3 details the full methodology, covering the dataset, sentence generation process, embedding models, autoencoder architectures, baseline methods, explainability integration, and evaluation metrics.

Chapter 4 presents and discusses results from both phases, including embedding comparisons, autoencoder benchmarking, and explainability analysis.

Chapter 5 summarizes the findings, evaluates how well the objectives were met, discusses limitations, and proposes directions for future research.

CHAPTER 2 LITERATURE REVIEW

2.1 Introduction

This chapter reviews existing research across the key areas that this project builds on: APT detection, anomaly detection methods, the use of LLMs and text embeddings in cybersecurity, autoencoder architectures, and the explainability techniques framework. The goal is to establish what the field currently knows, identify what has already been tried, and pinpoint the specific gaps that this project is designed to fill. The chapter ends with a summary of those gaps and how this work responds to each one.

2.2 Cyber Advanced Persistent Threats (APTs)

Advanced Persistent Threats are a distinct category of cyberattack. What sets them apart from ordinary attacks is not just their technical sophistication, but their patience. APT actors do not rush. They plan carefully, move through a target network in stages, and maintain access for extended periods, sometimes months or years, before anyone notices something is wrong. The typical APT lifecycle follows a well-documented sequence: initial reconnaissance, gaining access through spear-phishing or software exploits, escalating privileges, moving laterally across the network, establishing command-and-control (C2) communication, and finally exfiltrating data or causing disruption [6].

A key reason APTs are so difficult to detect is their use of tools that already exist on the target system. This strategy is called “living-off-the-land” (LotL). By abusing legitimate utilities like PowerShell, WMI, and scheduled tasks, attackers make their activity look like normal administrative work [7]. There are no unusual executables dropped, no suspicious downloads, just standard system tools being used in slightly unusual ways. This is what makes conventional detection approaches fundamentally ineffective against APTs.

The damage caused by APT campaigns has been well-documented. High-profile incidents have targeted government agencies, defense contractors, healthcare systems, and critical infrastructure operators. Threat intelligence reports from organizations like Mandiant and CrowdStrike consistently show that APT activity is increasing in both frequency and scope [8]. This escalating threat landscape is one of the main reasons the research community has shifted toward more intelligent, behavior-based detection frameworks that do not rely on knowing what an attack looks like in advance.

2.3 Anomaly Detection in Cybersecurity

Anomaly detection is built around a simple but powerful idea: learn what normal looks like, then flag anything that deviates significantly from it. In a cybersecurity context, this approach is attractive because it does not depend on knowing about a specific attack beforehand. If something unusual is happening, the system can flag it even if it has never seen that particular attack pattern before [5].

The most commonly used classical methods for anomaly detection are Isolation Forest, One-Class SVM (OCSVM), and PCA-based reconstruction. Isolation Forest works by randomly partitioning the data and scoring samples based on how quickly they become isolated. Anomalies tend to get isolated faster because they are different from most of the data [9]. OCSVM takes a different route: it learns a tight boundary around normal data in a transformed feature space and flags anything that falls outside that boundary as anomalous. PCA reduces the data to its most important dimensions and then measures how well each sample can be reconstructed from those dimensions. Samples that cannot be represented accurately by the normal structure of the data are treated as outliers.

These methods have been used successfully in network intrusion detection and log analysis. However, they hit a wall when applied to APTs for a specific reason. APT-related events are not always statistically extreme. A single log entry might look completely normal on its own. It is only when that entry is viewed alongside the events before and after it that the attack becomes visible. Classical anomaly detection methods evaluate each sample independently, which means they miss the sequential and contextual signals that APTs rely on to stay hidden [10].

This limitation has driven a shift toward deep learning-based anomaly detection, with autoencoders being the most widely studied approach. Autoencoders can learn complex, non-linear representations of normal behavior and detect subtle deviations more reliably than classical methods. A review by Mutalib et al. [10] of explainable deep learning approaches for APT detection found that autoencoder-based systems consistently reached between 96.7% and 98% accuracy across benchmark datasets, clearly outperforming classical methods when dealing with the multi-stage, sequential nature of APT attacks.

2.4 Large Language Models for Security Applications

Large Language Models have become increasingly relevant to cybersecurity over the past few years. Their ability to understand context, encode meaning across sequences,

and generalize to new domains makes them useful for security tasks that go well beyond simple pattern matching.

Ali et al. [8] surveyed LLM applications across eight cybersecurity domains, covering tasks such as vulnerability detection, malware analysis, intrusion detection, and phishing identification. The key finding was that LLMs offer something rule-based systems cannot: they reason about the meaning behind a sequence of events, not just whether it matches a known pattern. The survey also noted that encoder-only models like BERT and RoBERTa tend to perform best on classification tasks, while encoder-decoder models are more suitable for generation tasks like automated report writing or threat summarization.

Aziz et al. [11] focused specifically on LLM applications for APT detection. Their review concluded that models such as BERT, RoBERTa, GPT-3, and LLaMA2 show strong potential for automated threat intelligence extraction and anomaly detection. However, they raised two practical concerns. First, large models carry significant computational cost. Second, fine-tuning them for cybersecurity tasks is difficult because labelled APT data is rare and hard to obtain.

Jaffal et al. [12] conducted a broader systematic review covering 172 studies. While LLMs showed strong performance across security domains, the review identified persistent challenges around interpretability, robustness to adversarial inputs, and the absence of standardized evaluation benchmarks for cybersecurity. This reinforces a key argument driving this project: strong detection performance alone is not sufficient. LLM-based systems need to be paired with explainability mechanisms before they can be trusted in real security operations.

Multi-agent LLM architectures have also attracted attention as a way to improve detection capabilities. Rawat et al. [7] proposed a modular multi-agent system using LLaMA 3.3-70B with Retrieval-Augmented Generation (RAG) and Chain-of-Thought prompting. The system achieved 93.6% detection accuracy and reduced false positives by 41.3% compared to traditional approaches. These are impressive numbers, but the system is computationally expensive and relies on supervised signals, making it poorly suited for the unsupervised anomaly detection setting that APT detection typically requires.

2.5 Text Embeddings for Cybersecurity Log Analysis

The idea of converting system log events into natural language sentences and encoding them as vector embeddings is relatively new, but early results have been promising. The core reasoning is straightforward. LLM embeddings carry semantic and contextual information that raw numerical feature vectors do not. When a log entry is expressed as a sentence and passed through an LLM, the resulting vector reflects not just the individual event, but the broader behavioral context around it. That context is precisely what separates normal activity from malicious activity in an APT scenario.

The most directly relevant prior work comes from the APT-LLM framework [6], which this project extends. In that study, system provenance traces from the DARPA TC dataset were converted into natural language sentences and encoded using five pre-trained models: BERT, ALBERT, RoBERTa, DistilBERT, and MiniLM, producing embeddings ranging from 384 to 1024 dimensions. These embeddings were then passed through three autoencoder architectures: a Baseline AE, a VAE, and a DAE. The best combination, ALBERT paired with a VAE, achieved an AUC-ROC of 0.95 and clearly outperformed classical baselines including OC-SVM, DBSCAN, and Isolation Forest.

Zuo et al. [13] took a different but complementary approach. They used GPT-4o to generate enriched natural language descriptions of system events before encoding them with OpenAI’s `text-embedding-3-small` model. These descriptions added context that raw logs lack, covering system call behavior, software identity, and execution context. The enriched embeddings achieved 99.1% accuracy with an MLP classifier on the ProvSec dataset. Notably, this approach also solved the Out-Of-Vocabulary (OOV) problem that affects static embedding methods like Doc2Vec, which had only reached between 45% and 62% accuracy on the same data.

Together, these results suggest that the choice of embedding model matters enormously. However, no existing study has conducted a systematic, controlled comparison across a diverse range of embedding families, including encoder-based, decoder-based, angle-optimized, and static models, specifically on APT log data. Filling that gap is one of the central contributions of Phase 2 of this project.

2.6 Autoencoder Architectures for Anomaly Detection

An autoencoder is a neural network with a specific training objective: compress the input into a compact representation, then reconstruct the original input from that compressed form as accurately as possible. When trained only on normal data, the network

becomes good at reconstructing normal patterns but struggles to reconstruct anomalous ones. This reconstruction error becomes the anomaly score [12]. The higher the error, the more likely the input is anomalous.

The Vanilla Autoencoder is the simplest version of this design and serves as the natural starting point for comparison. Several architectural variants have been developed to address specific weaknesses of the basic design. The Sparse Autoencoder adds a constraint on the hidden layer activations, forcing the model to use only a small subset of neurons at any given time [14]. This encourages more selective and interpretable feature representations. The Denoising Autoencoder trains the model to reconstruct clean input from intentionally corrupted versions, which improves its ability to handle the noise commonly found in real-world log data [15].

The Variational Autoencoder (VAE) takes a probabilistic approach. Rather than encoding each input to a fixed point in latent space, it learns a distribution over possible representations [16]. This matters for anomaly detection because anomalies tend to fall in low-probability regions of the learned distribution, making them easier to identify. The β -VAE extends this by adding a tunable weight to the regularization term, which encourages the model to learn a more disentangled latent representation where individual dimensions correspond to distinct factors of variation. The GMVAE takes this further by replacing the single Gaussian prior with a Gaussian Mixture, allowing the model to capture multiple distinct modes of normal behavior [17]. This is particularly valuable in APT datasets where benign activity can be highly diverse, covering many different types of legitimate system operations.

Attention-based autoencoders incorporate self-attention mechanisms that allow the model to weigh the importance of different input features dynamically [18]. Rather than treating all features as equally relevant, the model learns to focus on the parts of the input that matter most. This is well-suited to log data where only a small number of fields may be informative at any given time. The LSTM Autoencoder processes input as a temporal sequence, which makes it naturally suited for log data where the order of events carries meaningful information. Afnan et al. [19] demonstrated the advantage of this kind of sequence-aware processing in their LogShield framework. Their transformer-based model, which captures sequential context through self-attention, outperformed LSTM baselines on DARPA datasets and achieved an F1-score of 98.4%.

Deep SVDD (Support Vector Data Description) works differently from reconstruction-based autoencoders [20]. Instead of measuring reconstruction error, it trains a neural network to map all normal data to a compact hypersphere in latent space. Anomalies

fall outside this sphere and are detected by their distance from the center. The Robust Autoencoder uses matrix decomposition to split the input into a low-rank component representing normal behavior and a sparse component capturing anomalous patterns. This makes it more resilient when the training data itself contains some contamination. The Contractive Autoencoder adds a regularization penalty based on the sensitivity of the encoder’s output to small changes in the input [21]. This makes the learned representations more stable and less likely to overfit to noise.

Despite how widely these architectures are studied in the general anomaly detection literature, no study has placed all of them in the same controlled experiment on APT-specific log data. This is a direct gap that Phase 2 of this project addresses.

2.7 Transformer-Based Approaches for APT Detection

Beyond autoencoders, transformer models have also been applied directly to APT detection. Rather than using transformers as feature extractors for downstream anomaly detectors, these approaches treat log sequences as inputs to full self-attention architectures.

Afnan et al. [19] developed LogShield, a three-layer detection framework. It converts provenance graph traces into structured log keys, then feeds those sequences into a RoBERTa-based transformer with a custom temporal embedding layer. The temporal embedding uses a sliding window to capture the timing patterns that are characteristic of APT’s slow, methodical progression through a network. Standard position-based embeddings miss this entirely because they encode position in a sequence but not the actual time between events. On the DARPA OpTC dataset, which has an extreme class imbalance of just 0.0016% malicious events, LogShield reached an F1-score of 98.4%, outperforming both LSTM and BERT baselines.

Aly et al. [22] proposed OCR-APT, which combines a graph convolutional network for detecting anomalies at the subgraph level with a GPT-4o-based investigation layer that automatically generates structured attack reports aligned to the APT kill chain. On the DARPA TC3 and OpTC datasets, it achieved an average F1-score of 0.96. The system uses RAG and Chain-of-Thought prompting to reduce hallucinations in the LLM layer, which is a genuine methodological challenge when using large generative models for security reporting. That said, running GPT-4o at inference time carries significant computational cost, which limits how practical this approach is in real-world deployments.

These two systems confirm that modeling log data as sequences, whether through temporal embeddings or graph-based representations, provides a real advantage over treating logs as flat feature vectors. They also highlight the trade-off between model capability and practical deployability, which is part of why the embedding-based, unsupervised approach taken in this project remains relevant. It is lighter, does not require labelled data, and can be systematically evaluated across a wide range of configurations.

2.8 Explainability in APT Detection

Explainability is not just an academic concern in AI-driven security systems. It is a practical requirement. An analyst who receives an alert but has no way of understanding why it was raised cannot act on it confidently. This is especially true in high-stakes environments like security operations centers, where false positives carry real costs and missed threats can have serious consequences. Beyond operational necessity, regulatory frameworks in critical sectors are also beginning to require that automated decisions be explainable and auditable [14].

SHAP (SHapley Additive exPlanations) is currently one of the most widely used methods for making model predictions interpretable. It assigns each input feature a contribution score based on game-theoretic Shapley values, which guarantees that the contributions are both consistent and locally accurate [23]. In practice, SHAP tells the readers exactly how much each feature pushed the model’s output up or down for a given prediction. Hasan et al. [15] applied SHAP alongside XGBoost on the SCVIC-APT-2021 dataset and found that features like “Idle Max” and “Fwd Header Length” were the strongest predictors of specific APT activity classes. This kind of feature-level insight is directly useful for alert triage because it tells an analyst where to look first.

Mutalib et al. [10] reviewed 100 studies on explainable deep learning approaches for APT detection and found a consistent pattern: models with integrated explainability led to higher analyst trust and faster incident response compared to black-box alternatives. Their review also confirmed that using SHAP and LIME together provides complementary perspectives that neither method offers alone.

Despite this evidence, integrating SHAP into embedding-based autoencoder pipelines for unsupervised APT detection remains largely unexplored. Most existing work applies these methods to supervised classifiers with interpretable input features. Applying them to high-dimensional LLM embeddings in an unsupervised setting introduces new challenges around what “features” even mean and how attribution should be interpreted.

This project addresses that gap directly.

2.9 Research Gaps

The literature reviewed above reveals four specific gaps that directly motivate this project.

No systematic embedding model comparison for APT data. Studies like APT-LLM [6] and the knowledge transfer work of Zuo et al. [13] each evaluate a small, fixed set of embedding models chosen without a systematic rationale. No study has conducted a broad, controlled comparison across encoder-based, decoder-based, angle-optimized, and static embedding families on APT log data specifically. Without this comparison, practitioners have no empirical basis for choosing which embedding model to use.

Limited autoencoder architecture diversity. Most existing studies compare two or three autoencoder variants at most. The full spectrum of architectures, including probabilistic models like GMVAE, sequence-aware models like LSTMAE, and robustness-oriented designs like the Contractive and Robust Autoencoder, has never been benchmarked together in a single controlled experiment on APT data. This leaves significant uncertainty about which architectural choices actually matter and why.

Explainability is rarely applied to deep unsupervised detectors. SHAP have been applied effectively to supervised classifiers [10], but their integration into embedding-based autoencoder pipelines for unsupervised anomaly detection is largely unexplored. Moving from interpretable input features to high-dimensional LLM embeddings changes the nature of the attribution problem entirely, and this has not been adequately addressed in the existing literature.

This project addresses all three gaps within a single unified framework, building incrementally from the validated Phase 1 baseline toward a detection system that is comprehensive, interpretable, and actionable. The tabulated literature review is shown in the Table 2.1.

Table 2.1: Summary of Literature Related to APT Detection, LLMs, Autoencoders, and Explainability

Study	Research Area	Dataset	Key Findings	Research Gap / Limitation
Benabderrahmane et al. [6]	LLM Embeddings + Autoencoders	DARPA TC	ALBERT + VAE achieved AUC-ROC of 0.95 and outperformed classical anomaly detection methods.	Limited embedding models and autoencoder architectures evaluated.
Ali et al. [8]	LLMs in Cybersecurity	Survey	LLMs demonstrated strong performance across multiple cybersecurity domains.	Challenges remain in explainability and real-world deployment.
Aziz et al. [11]	LLMs for APT Detection	Survey	BERT, RoBERTa, GPT, and LLaMA showed strong potential for threat intelligence extraction.	High computational requirements and scarcity of labelled datasets.
Jaffal et al. [12]	LLMs for Cybersecurity	172 Studies	Identified effectiveness of LLMs across multiple security applications.	Interpretability, robustness, and benchmark standardization remain unresolved.
Rawat et al. [7]	Multi-Agent LLM	APT Dataset	Achieved 93.6% accuracy and reduced false positives by 41.3%.	Computationally expensive and relies on supervised learning.

Continued on next page

Table 2.1 – Continued from previous page

Study	Research Area	Dataset	Key Findings	Research Gap / Limitation
Zuo et al. [13]	Knowledge-Enriched Embeddings	ProvSec	GPT-4o generated contextual descriptions achieved 99.1% accuracy.	Focused on supervised classification rather than unsupervised anomaly detection.
Mutalib et al. [10]	Explainable APT Detection	Review	Explainability improved analyst trust and incident response times.	Limited work exists on explainable unsupervised deep learning systems.
Afnan et al. [19]	Transformer-Based Detection	DARPA	LogShield achieved 98.4% F1-score using temporal-aware transformers.	Focused exclusively on transformer models without autoencoder comparisons.
Aly et al. [22]	Graph + LLM Detection	DARPA TC3, OpTC	OCR-APT achieved an average F1-score of 0.96.	Requires expensive GPT-4o inference and significant computational resources.
Lundberg and Lee [23]	SHAP Explainability	General ML	Provided theoretically grounded feature attribution using Shapley values.	Application to high-dimensional LLM embeddings remains challenging.

Continued on next page

Table 2.1 – Continued from previous page

Study	Research Area	Dataset	Key Findings	Research Gap / Limitation
Chandola et al. [5]	Anomaly Detection	Survey	Established foundational anomaly detection concepts and taxonomy.	Does not address deep learning or sequential APT data.
Liu et al. [9]	Anomaly Detection	Benchmark Data	Introduced Isolation Forest for efficient outlier detection.	Evaluates data independently, missing sequential APT context.
Hinton & Salakhutdinov [14]	Autoencoders	General ML	Demonstrated neural networks for effective dimensionality reduction.	General machine learning focus, not tailored to cybersecurity logs.
Vincent et al. [15]	Denoising Autoencoders	Image/Text	Showed DAEs learn robust features by reconstructing noisy inputs.	Not systematically applied to LLM embeddings of system logs.
Kingma & Welling [16]	Variational Autoencoders	Image Data	Introduced VAEs for probabilistic latent space modeling.	Focused on data generation rather than log anomaly detection.
Dilokthanakul et al. [17]	GMVAE	General ML	Extended VAEs with Gaussian Mixture priors for multi-modal data.	Application to diverse APT log behavior remains untested.
Ruff et al. [20]	Deep SVDD	Image Data	Mapped normal data to a hypersphere for one-class classification.	Effectiveness on high-dimensional text embeddings is unknown.

Continued on next page

Table 2.1 – Continued from previous page

Study	Research Area	Dataset	Key Findings	Research Gap / Limitation
Rifai et al. [21]	Contractive Autoencoders	General ML	Made feature representations robust to small changes in input.	Not benchmarked against other autoencoders for APT detection.

CHAPTER 3 RESEARCH METHODOLOGY

3.1 Introduction

This chapter describes the full methodology used to build, train, and evaluate an explainable deep learning pipeline for detecting Advanced Persistent Threats (APTs) in system audit logs. One of the central challenges in applying deep learning to cybersecurity is that neural networks are inherently opaque [24]. They produce anomaly scores without explaining what triggered them. To address this, the research proposes a dual-pipeline architecture that combines high-dimensional semantic anomaly detection with token-level explainability, so that every flagged alert can be traced back to specific words and actions in the original log. This chapter covers the data handling protocols, the benchmarking of embedding models and Autoencoder architectures, the end-to-end detection pipeline, and the explainability frameworks applied at both the token and latent space levels. Figure 3.1 provides an overview of the full end-to-end architecture.

3.2 Dataset Acquisition, Standardization, and Anti-Leakage Protocol

The dataset used in this project comes from the ADAPT repository, which stands for Audit Data for APT Analysis. ADAPT was built specifically to support research on APT detection using system audit logs. It provides a controlled and reproducible environment where real APT attack scenarios are simulated and all system activity is captured at the audit log level.

The underlying data originates from the DARPA Transparent Computing (TC) Engagement 3 (E3) program [25], which collected system telemetry from five different monitoring performers: 5DIR, CADETS, CLEARSCOPE, THEIA, and TRACE. These performers ran on four different operating systems, BSD, Linux, Windows, and Android, and captured events across many different machines. The behavioral contexts captured include `ProcessAll`, `ProcessExec`, `ProcessEvent`, `ProcessNetflow`, and `ProcessParent`, each representing a different dimension of system activity [26].

Each record in the dataset captures a single system-level event. This could be a process execution, a file read or write, a network connection, or a registry modification. Each event comes with associated metadata including the subject entity, the object entity, the action type, and a timestamp [27]. Records are labeled either as benign, representing normal day-to-day system operations, or malicious (APT), representing

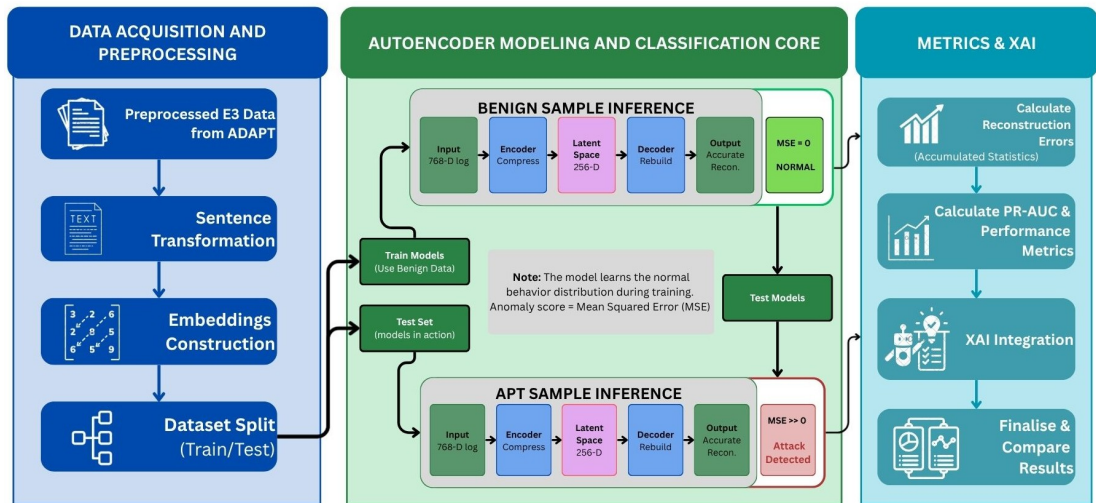


Figure 3.1: Overall System Architecture

events generated during APT attack simulations. Table 3.1 summarizes the key statistics of the ADAPT dataset used in this project.

Table 3.1: Experimental datasets of DARPA’s TC program used in our study. A dataset entry (columns 3 to 7) is described by the number of rows (processes) / the number of columns (attributes). For instance, with ProcessAll (PA) obtained from the second scenario of Linux, the dataset comprises 279,369 rows and 457 attributes, including 100 APT attacks (0.036%).

	Size	PE	PX	PP	PN	PA	<i>nb_attacks</i>	$\% \frac{nb_attacks}{nb_processes}$
BSD	265 MB	224624 / 32	224146 / 136	223780 / 38	42888 / 63	224624 / 266	11	0.005
Windows	36.0 MB	11151 / 31	11077 / 372	10992 / 81	329 / 126	11151 / 607	9	0.081
Linux (1)	3.63 GB	282087 / 26	271088 / 141	263730 / 46	6580 / 6226	282103 / 6436	46	0.016
Linux (2)	465 MB	270652 / 19	278363 / 160	279090 / 71	48048 / 210	279369 / 457	100	0.036
Android	18.1 MB	12106 / 28	12106 / 45	24 / 12	1116 / 214	12106 / 296	13	0.107

One feature of this dataset that immediately stands out is the extreme class imbalance. Across all operating system scenarios, malicious events make up a tiny fraction of total records, ranging from just 0.005% in the BSD scenario to 0.107% in the Android scenario. This is not a data collection artifact. It accurately reflects real-world APT environments, where attackers operate slowly and carefully to minimize the footprint of their activity. This imbalance directly shapes the experimental design, particularly the dataset splitting strategy described below.

To maintain consistency in entity tracking and prepare the data for downstream threat hunting, a standardization step was applied to the primary identifiers. Legacy `uuid` columns, which varied in format across sources, were renamed and standardized to a universal `Object_ID` schema. This ensures that processes and network connections can be accurately correlated across different parts of the dataset.

To prevent data leakage, meaning a situation where the model inadvertently encounters malicious examples during training, a strict 80/20 train-test split was enforced based on ground-truth labels, where 0 indicates a benign event and 1 indicates an APT anomaly [28].

- **Training Set:** Consists of exactly 80% of the benign system logs. No anomalous records are included. This ensures the Autoencoder is trained exclusively on normal behavior, establishing a clean baseline with zero contamination from attack data.
- **Testing Set:** Consists of the remaining 20% of benign logs combined with 100% of the anomalous logs. This guarantees the model is evaluated against attack vectors it has never seen before, which is a faithful simulation of real-world deployment conditions.

Figure 3.2 visualizes the split logic used in the research. To ensure a fair evaluation, all anomalous records are completely excluded from the training phase and reserved for testing only. This protocol allows the models to learn normal behavioral patterns from benign data while accurately measuring their ability to identify previously unseen anomalous activities during inference.

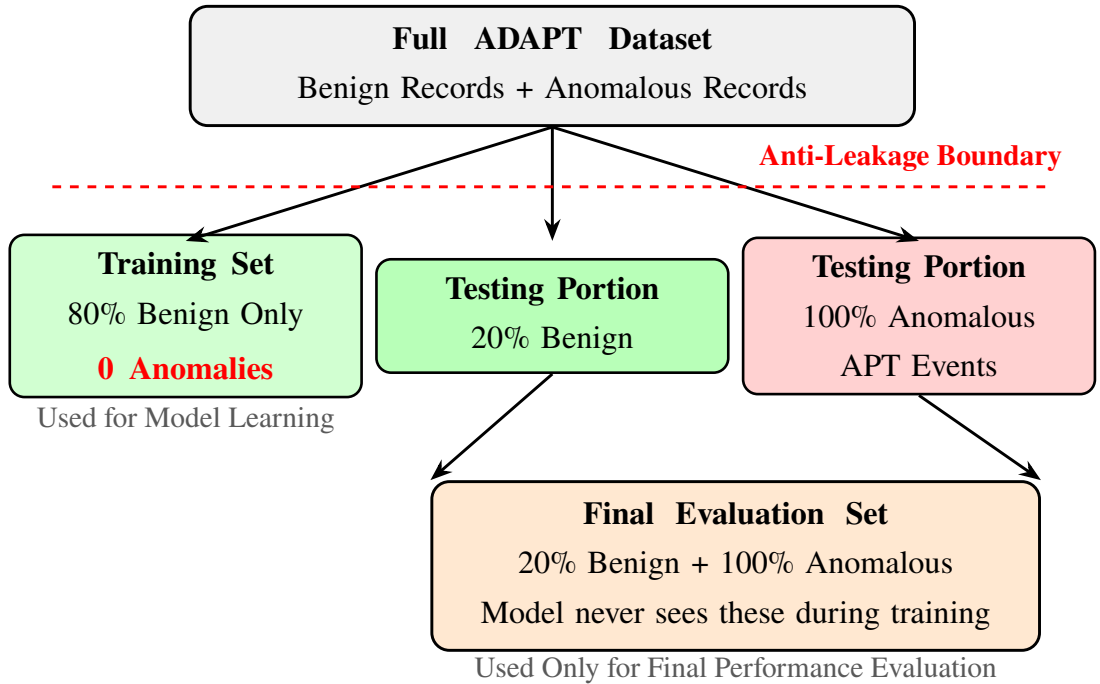


Figure 3.2: Improved anti-leakage 80/20 dataset split protocol. The model is trained exclusively on benign records, while all anomalous records are reserved for testing. This ensures unbiased evaluation and prevents data leakage.

3.3 Embedding Model Taxonomy and Benchmarking

A major part of this methodology involved benchmarking a broad range of Natural Language Processing (NLP) embedding models to find the best way to represent cybersecurity logs as numerical vectors. The goal was to identify which type of embedding captures the behavioral context of system activity most effectively, since the quality of the embedding directly determines how well the downstream Autoencoder can separate normal from anomalous behavior. The models are organized as follows, and fully detailed in Table 3.2.

The evaluated models were organized into five architectural groups. This grouping reflects genuine differences in how these models process text, and it allowed the benchmarking to assess not just which specific model performs best, but which architectural approach is most suitable for this type of data.

3.3.1 Group A: The “Encoder” Family (BERT-Based)

Encoder models process input text in both directions simultaneously, meaning each token in a sentence can attend to every other token. This bidirectional context modeling makes them particularly effective at capturing relationships within structured log syntax, such as the relationship between a process name, its arguments, and its parent process [29]. The models evaluated in this group were:

- BAAI/bge-m3 [30]
- Alibaba-NLP/gte-large-en-v1.5 [31,32]
- sentence-transformers/all-mpnet-base-v2 [33]
- facebook/contriever [34]
- jinaai/jina-embeddings-v3 [35]
- nomic-ai/nomic-embed-text-v1.5 [36]
- answerdotai/ModernBERT-large [37]
- intfloat/multilingual-e5-large [38]

3.3.2 Group B: The “Decoder” Family (LLM-Based)

Decoder-only models were originally designed for text generation. They process tokens sequentially from left to right rather than bidirectionally. Recently, these models have been adapted to produce embeddings by leveraging the rich, high-level representations they build up across their many layers. Their large parameter counts and emergent reasoning capabilities make them interesting candidates for complex log data, even though generation was not their original purpose. The models evaluated in this group were:

- `nvidia/NV-Embed-v2` [39,40]
- `Alibaba-NLP/gte-Qwen2-7B-instruct` [32]
- `intfloat/e5-mistral-7b-instruct` [41,42]
- `Salesforce/SFR-Embedding-Mistral` [43]
- `linq-ai/linq-embed-mistral` [44]
- `McGill-NLP/LLM2Vec-Meta-Llama-3-8B-Instruct-mntp` [45]

3.3.3 Groups C, D, and E: “Large”, “Angle”, and “Static” Architectures

Three additional paradigms were included to ensure the benchmarking was comprehensive.

- **Group C (Large Models):** Scaled architectures with high parameter counts, such as `sentence-transformers/gtr-t5-large` [46]. These models trade computational cost for representation capacity.
- **Group D (Angle-Optimized Models):** Models like `WhereIsAI/UAE-Large-V1` [47] that use angle-optimized contrastive learning. Standard contrastive training can cause embedding distributions to collapse or degrade at the extremes of the representation space. Angle optimization addresses this by regularizing the geometry of the learned space.
- **Group E (Static Embeddings):** Lightweight models such as `dunzhang/stella_en_400M_v5` [48] that produce fixed representations without contextual adjustment. These are much faster to run and serve as an important efficiency baseline.

Table 3.2: Taxonomy of LLM embedding models evaluated in this study, organized by architectural group.

#	Model	Group	Dim.	Key Characteristic
1	facebook/contriever [49]	A – Encoder	768	Unsupervised dense retriever
2	BAAI/bge-m3 [30]	A – Encoder	1024	Multilingual multi-task
3	gte-large-en-v1.5 [38]	A – Encoder	1024	Semantic similarity
4	all-mpnet-base-v2 [33]	A – Encoder	768	Strong sentence-level baseline
5	jina-embeddings-v3 [35]	A – Encoder	1024	Long-context (8192 tokens)
6	nomic-embed-text-v1.5 [36]	A – Encoder	768	Open-source, large context
7	ModernBERT-large [37]	A – Encoder	1024	Extended context BERT variant
8	multilingual-e5-large [38]	A – Encoder	1024	Extended XLM-RoBERTa framework
9	nvidia/NV-Embed-v2 [39,40]	B – Decoder	4096	Instruction-tuned decoder
10	gte-Qwen2-7B-instruct [32]	B – Decoder	3584	7B instruction-tuned decoder
11	e5-mistral-7b-instruct [41,42]	B – Decoder	4096	Mistral embedding model
12	SFR-Embedding-Mistral [43]	B – Decoder	4096	Retrieval-optimized mistral
13	linq-embed-mistral [44]	B – Decoder	4096	Instruction-tuned semantic search
14	LLM2Vec-Llama-3-8B-mntp [45]	B – Decoder	4096	LLaMA-3 bidirectional encoder
15	gtr-t5-large [46]	C – Large	768	T5 encoder-decoder
16	WhereIsAI/UAE-Large-V1 [47]	D – Angle-optimized	1024	Angle-based contrastive
17	stella_en_400M_v5 [48]	E – Static	1024	Lightweight, fixed repr.

To analyze how these different dimensionality settings and architectural choices influence the separation of system log activities, high-dimensional raw embeddings are visualized using t -Distributed Stochastic Neighbor Embedding (t -SNE). Figure 3.3 shows the t -SNE projections of log vectors before they are processed by the anomaly detection pipeline. For visualization, the authors use the best-performing combination of the embedding model and autoencoder. This helps us clearly observe whether certain model setups naturally separate anomalous log patterns from normal (benign) sequences in the original latent space.

3.4 Deep Learning Architecture: Autoencoder Topology Search

The Autoencoder is the core detection engine in this pipeline. It works by learning to compress normal log data into a compact internal representation and then reconstruct the original input from that compressed form. When the model is shown normal data, it reconstructs it well. When it is shown anomalous data that it has never seen during training, the reconstruction fails, and the resulting error becomes the anomaly score.

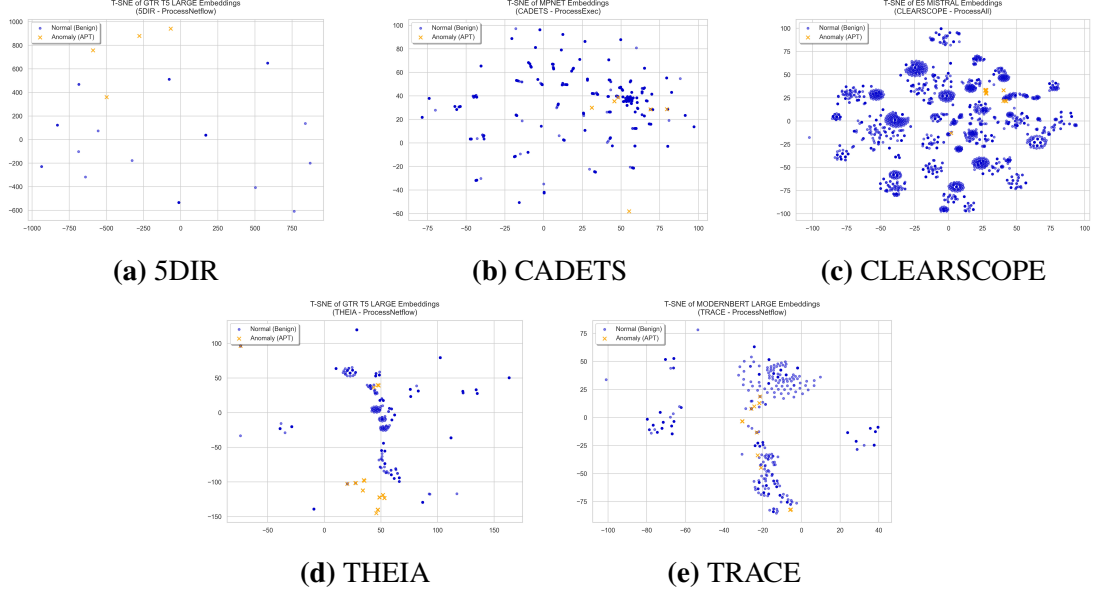


Figure 3.3: Two-dimensional t -SNE projection comparison of representative raw embedding groups. Malicious logs are marked in red while normal activities are marked in blue, demonstrating the initial cluster density before neural autoencoder filtering.

Formally, let $x \in \mathbb{R}^d$ represent the input vector, whether a TF-IDF row or a Contriever embedding. The encoder function f_θ , parameterized by weights and biases θ , compresses x into a lower-dimensional latent representation $h \in \mathbb{R}^k$, where $k < d$:

$$h = f_\theta(x) = \sigma(Wx + b) \quad (3.1)$$

Here, W is the encoder weight matrix, b is the bias vector, and σ is a non-linear activation function such as Rectified Linear Unit (ReLU). The decoder function g_ϕ , parameterized by ϕ , then maps the latent representation h back to the original input space to produce the reconstruction $\hat{x}$:

$$\hat{x} = g_\phi(h) = \sigma'(W'h + b') \quad (3.2)$$

Both the encoder and decoder parameters (θ, ϕ) are optimized together during training to minimize the difference between the original input and the reconstruction [50]. Since the input vectors are continuous, Mean Squared Error (MSE) is used as the training objective:

$$\mathcal{L}_{MSE}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^N ||x_i - g_{\phi}(f_{\theta}(x_i))||^2 \quad (3.3)$$

Because the Autoencoder is trained exclusively on benign system logs, it learns to minimize this error for normal behavior only. During inference, when an APT-related log vector x_{apt} is passed through the model, the network fails to reconstruct it accurately because the malicious patterns fall outside what it has learned to represent. This produces an elevated MSE score, which directly serves as the anomaly indicator for threshold-based classification as shown in Figure 3.4.

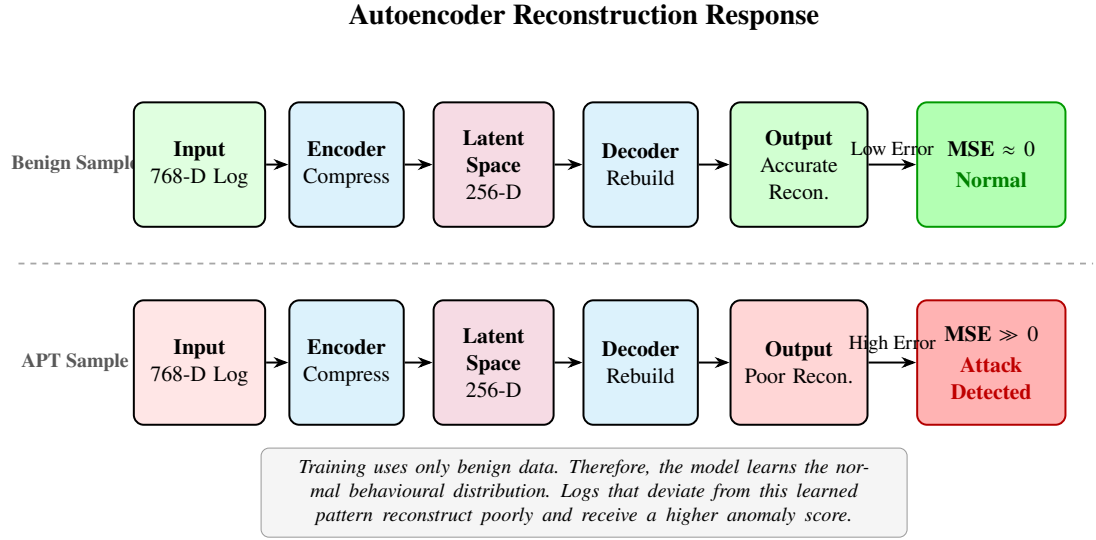


Figure 3.4: Enhanced visualization of autoencoder reconstruction behaviour for benign and APT logs. Benign samples align with the learned normal distribution and reconstruct with minimal error. APT samples fall outside this distribution, causing poor reconstruction and elevated mean squared error (MSE), which is used as the anomaly score.

Because APT attack patterns are highly varied, a suitable architecture was used to find the best-performing Autoencoder design. Eleven distinct architectures were implemented and evaluated for their reconstruction quality and latent space stability:

1. **Baseline (Vanilla) AE:** A symmetric, fully connected feedforward network. Serves as the reference point for all comparisons.
2. **Sparse AE:** Adds an L1 penalty to the hidden layer activations, forcing the model to activate only the most informative neurons for any given input.
3. **Deep SVDD:** Does not use reconstruction error. Instead, it trains a neural network to map all normal data into a compact hypersphere in latent space, with anomalies falling outside it.

4. **Denoising AE:** Trains the model to recover clean log representations from deliberately corrupted inputs, improving robustness to the noise commonly found in real system telemetry.
5. **Variational AE (VAE):** Learns a probability distribution over the latent space rather than a fixed point encoding. Anomalies tend to fall in low-probability regions of this distribution.
6. **β -VAE:** Extends the VAE with a tunable weight on the regularization term, encouraging the model to learn a more structured latent space where individual dimensions correspond to distinct properties of the input.
7. **GMVAE:** Replaces the single Gaussian prior of the VAE with a mixture of Gaussians, allowing the model to represent multiple distinct modes of normal behavior. This matters in APT datasets where benign activity spans many different types of legitimate system operations.
8. **Attention AE:** Incorporates self-attention mechanisms so the model can dynamically focus on the most relevant parts of each input, rather than treating all features as equally important.
9. **LSTMAE:** Processes log sequences as temporal data, capturing the ordering and timing relationships between events rather than treating each log entry as an independent vector.
10. **Robust AE:** Decomposes each input into a low-rank component representing normal background activity and a sparse component representing potential anomalies. This makes it more resilient when training data contains some level of contamination.
11. **Contractive AE:** Adds a regularization term that penalizes the sensitivity of the encoder's output to small changes in the input, producing more stable representations that generalize better to unseen data.

Explainability methods like SHAP work by measuring small, controlled changes in the model's output in response to changes in its input. This requires the model's behavior to be consistent and predictable. The stochastic sampling in VAEs introduces variability that makes these fine-grained measurements unreliable, undermining the entire purpose of the explainability pipeline.

To identify the best-performing architecture, 11 distinct Autoencoder topologies were implemented and evaluated. Table 3.3 summarizes each architecture, its mathematical regularization strategy, and whether it is deterministic or probabilistic. This classification is significant because, as discussed, the explainability pipeline requires deterministic behavior to produce stable SHAP attributions.

Table 3.3: Autoencoder topologies evaluated in the architecture search, with their regularization strategies and determinism properties.

Topology	Regularization / Objective	Deterministic
Baseline (Vanilla) AE [14]	Standard MSE reconstruction loss	Yes
Sparse AE [51]	L1 sparsity penalty on hidden activations	Yes
Denoising AE (DAE) [52]	Reconstruct clean input from corrupted version	Yes
Robust AE [53]	Low-rank and sparse matrix decomposition	Yes
Contractive AE [21]	Frobenius norm penalty on encoder Jacobian	Yes
Attention AE [54]	Self-attention weighting on input features	Yes
LSTM AE [55]	Sequential temporal modeling via LSTM layers	Yes
Deep SVDD [56]	Hypersphere boundary without reconstruction	Yes
Variational AE (VAE) [57]	KL-divergence to Gaussian prior	No
β -VAE [58]	Weighted KL-divergence for disentangled latent space	No
GMVAE [59]	Gaussian mixture prior with multimodal latent space	No

Table 3.4 shows the corresponding network architectures, key hyperparameters, and optimization objectives used during training. All models were trained under identical optimization settings to ensure a fair comparison across architectures.

Table 3.4: Autoencoder architectures and hyperparameters used during the architecture search.

Model	Base Architecture	Key Hyperparameters	Objective Function
Baseline AE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	N/A	MSE
Sparse AE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	Sparse penalty = 10^{-4}	MSE + L1 penalty
Deep SVDD	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 128)	Latent dimension = 128	Distance to hypersphere center
DAE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	Noise factor = 0.2	MSE
VAE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	Latent dimension = 256	MSE + KLD
Beta-VAE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	$\beta = 4.0$	MSE + $\beta \times$ KLD
GMVAE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	Components = 5	MSE
Attention AE	Multi-head attention	Attention heads = 8	MSE
LSTM AE	LSTM (Input $\rightarrow$ 256)	Hidden dimension = 256	MSE
Robust AE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	N/A	L1 loss (MAE)
Contractive AE	Linear (Input $\rightarrow$ 1024 $\rightarrow$ 256)	$\lambda = 10^{-4}$	MSE + Jacobian penalty

Note: All autoencoders were trained using the Adam optimizer with a learning rate of 0.001, batch size of 128, and 15 epochs.

3.5 The End-to-End Anomaly Detection Pipeline

The full detection pipeline moves through four sequential stages, taking raw system telemetry as input and producing scored, explainable alerts as output [6].

1. **Ingestion and Preprocessing:** Raw system log records are loaded, null values are removed, and structural metadata fields are standardized across all data sources.

This step ensures the input to the pipeline is clean and consistently formatted.

2. **Latent Projection:** Each preprocessed log record is converted into a natural language sentence and passed through the Contriever tokenizer and embedding model. Attention-weighted mean pooling is applied to the token outputs, followed by L2 normalization. The result is a dense 768-dimensional vector that captures the semantic and behavioral context of the original log entry.
3. **Autoencoder Inference:** The 768-dimensional embedding vector is passed through the trained Vanilla Autoencoder. The encoder compresses it down to a 256-dimensional bottleneck representation, and the decoder attempts to reconstruct the original 768-dimensional vector from that compressed form.
4. **Thresholding and Scoring:** The MSE between the original input vector and the reconstructed output is calculated. If this error exceeds a statistically derived threshold, the log entry is flagged as a potential APT event and passed to the XAI framework for further analysis and triage.

3.6 The Dual-Pipeline XAI Attribution Framework

Flagging an anomaly is only the first step. For an alert to be actionable, an analyst needs to understand what in the log caused the elevated anomaly score. The XAI framework achieves this through two complementary pipelines, each operating at a different level of the detection system.

3.6.1 Pipeline 1: Token-Level Explainability (TF-IDF and SHAP)

The first pipeline operates on a TF-IDF representation of the log text rather than on LLM embeddings. TF-IDF maps every unique word in the log corpus to a distinct feature dimension, which means each feature has a clear, human-readable identity. A SHAP Explainer [23] is then applied to the TF-IDF Autoencoder. SHAP uses cooperative game theory to measure how much each individual token contributes to the model's output. The result is a set of exact numerical impact values assigned to specific words in the original log, directly identifying which tokens, such as `taskkill` or a suspicious directory path, drove the anomaly score upward [60].

3.6.2 Pipeline 2: LLM Latent Space Ablation Study

Applying SHAP directly to model embeddings does not produce interpretable results, because individual embedding dimensions do not correspond to meaningful human-

readable concepts. To bridge this gap, an Automated Multi-Stage Ablation Study was designed to probe the LLM’s internal representations in a structured way as shown in Figure 3.5.

The process works as follows. The top threat tokens identified by the TF-IDF SHAP pipeline are extracted. The original log text is then passed through the full pipeline to compute the baseline MSE. Next, the identified threat tokens are systematically removed. The removed text is re-embedded, and the new MSE score is recorded. The difference between the two scores provides a direct, mathematically grounded measurement of how much those specific tokens contributed to the anomaly detection in the LLM embedding space. When threat tokens are removed, the embedding shifts predictably toward the benign distribution, and the anomaly score drops. This is not an approximation. It is a controlled experiment that directly links human-readable log content to the model’s internal behavior.

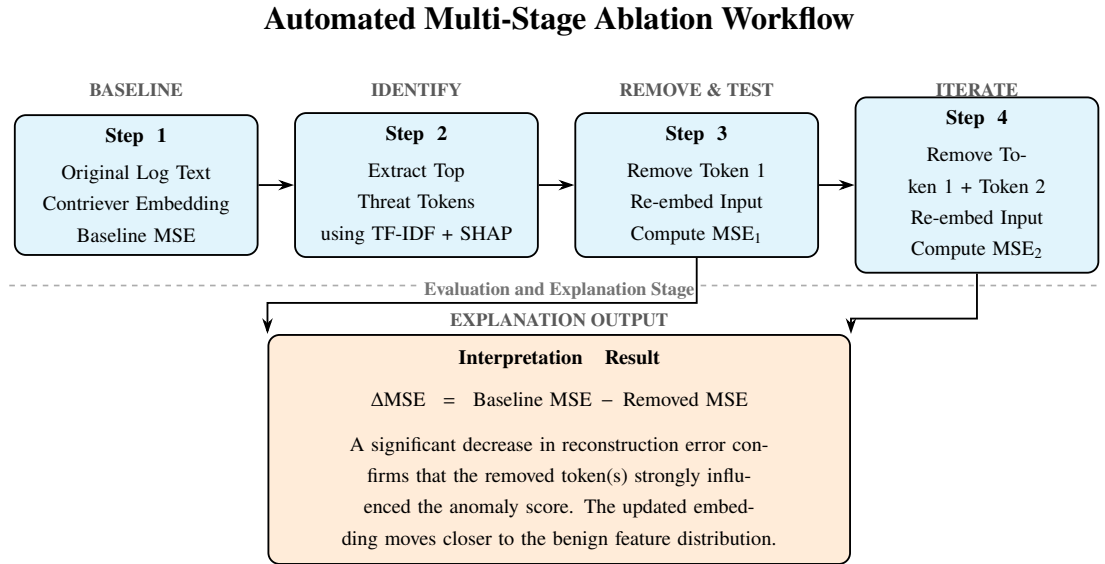


Figure 3.5: Enhanced automated multi-stage ablation workflow with improved alignment and flow structure. Threat-indicative tokens identified by TF-IDF SHAP are progressively removed and re-evaluated. The resulting reduction in reconstruction error quantitatively reveals which tokens most strongly drive the anomaly signal.

3.7 Evaluation Metrics and Experimental Setup

The performance of the pipeline is measured using metrics chosen specifically for the challenges of APT detection, where anomalies are rare and the cost of both missed detections and false alarms is high.

- **Reconstruction Error (MSE):** This serves as the continuous anomaly score. For an input vector x of dimension D and its reconstruction $\hat{x}$, MSE is defined

as:

$$\text{MSE}(x, \hat{x}) = \frac{1}{D} \sum_{i=1}^D (x_i - \hat{x}_i)^2 \quad (3.4)$$

A higher MSE indicates the model struggled to reconstruct the input, which signals a likely anomaly.

- **Area Under the Receiver Operating Characteristic Curve (AUC-ROC):** This metric evaluates how well the model separates anomalies from normal events across all possible classification thresholds. It plots the True Positive Rate (TPR) against the False Positive Rate (FPR) and measures the area under that curve:

$$\text{AUC-ROC} = \int_0^1 \text{TPR} \left(\text{FPR}^{-1}(t) \right) dt \quad (3.5)$$

where $\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}$ and $\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}$. A score of 1.0 represents perfect separation, while 0.5 represents random guessing [61].

- **Precision-Recall Area Under the Curve (PR AUC):** Because APT datasets are severely imbalanced, AUC-ROC alone can be misleading. A model that flags everything as normal would still achieve a high AUC-ROC simply because normal events dominate. PR AUC addresses this by focusing specifically on the model's performance on the minority anomaly class. It measures the area under the Precision-Recall curve:

$$\text{PR-AUC} = \int_0^1 P(R) dR \quad (3.6)$$

where Precision is $P = \frac{\text{TP}}{\text{TP} + \text{FP}}$ and Recall is $R = \frac{\text{TP}}{\text{TP} + \text{FN}}$. A high PR AUC means the model is identifying real anomalies without generating excessive false alarms [62].

All experiments were implemented in Python and executed on NVIDIA GPUs, specifically an RTX 1080 Ti (11GB VRAM) and an RTX 5060 Ti (16GB VRAM), using CUDA acceleration. Core libraries used throughout the project include `torch`, `transformers`, `shap`, and `scikit-learn`.

CHAPTER 4 RESULTS AND DISCUSSION

4.1 Experimental Overview

This chapter presents all experimental results from the full benchmarking study and discusses their meaning. The evaluation covers 17 pre-trained LLM embedding models paired with 11 autoencoder architectures, producing 176 unique combinations per context per performer. These were evaluated across five DARPA TC performers (5DIR, CADETS, CLEARSCOPE, THEIA, and TRACE) and five process contexts (ProcessAll, ProcessEvent, ProcessExec, ProcessNetflow, and ProcessParent). Across the full study, this gives a total of 4,400 individual experiment runs.

Because the dataset is severely class-imbalanced, with APT events making up as little as 0.005% of total records in some performers, PR-AUC is the primary evaluation metric throughout this section. AUC-ROC rewards a model for correctly ranking normal events above anomalous ones across all thresholds, meaning a model can achieve a high AUC-ROC simply by being good at classifying the overwhelming majority class. PR-AUC directly measures how precisely the model identifies the rare positive APT class and how many of those rare events it actually catches. For a real SOC environment where false alarm fatigue is a genuine operational problem, PR-AUC is the honest metric.

All results were produced under a strict anti-leakage protocol. Ground truth labels were never used during training. The model only ever saw benign data during training and was evaluated on a test set containing the held-out 20% of benign samples combined with 100% of the APT events it had never encountered before.

4.2 ProcessAll Context: Best Combinations Per Performer

The ProcessAll context combines all process-level event types into a single unified log representation, capturing process executions, file accesses, network connections, and registry modifications together. It is the broadest context in the evaluation and the natural starting point for the analysis. Table 4.1 shows the best embedding and autoencoder combination for each performer under this context.

There is an immediate and clear split in these results. Clearscope, THEIA, and TRACE all achieve PR-AUC above 0.83. The 5DIR and CADETS performers fall well below this at 0.39 and 0.48 respectively. Understanding this gap requires looking at the data

Table 4.1: Best embedding and autoencoder combination per performer under the ProcessAll context, ranked by PR-AUC.

Performer	Best Combination	PR-AUC
5DIR	gte-Qwen2-7B-instruct + Contractive AE	0.3939
CADETS	BAAI/bge-m3 + LSTMAE	0.4839
CLEARSCOPE	SFR-Embedding-Mistral + Deep SVDD	0.8682
THEIA	multilingual-e5-large + Baseline AE	0.8309
TRACE	gtr-t5-large + Baseline AE	0.8377

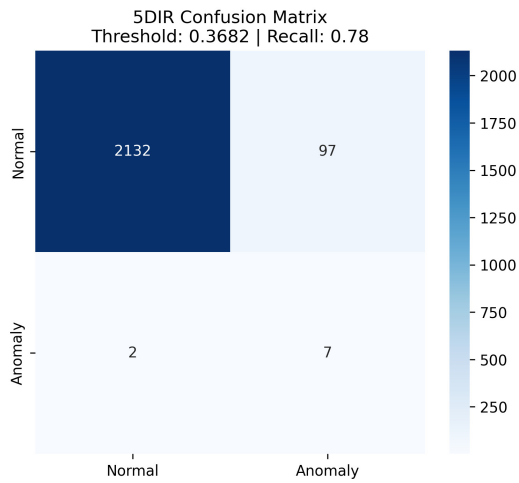
rather than the models.

The 5DIR performer runs on BSD and contains only 11 malicious events across the entire dataset. Evaluating PR-AUC on a positive class of 11 samples is inherently fragile. Missing two detections moves the recall by nearly 20 percentage points. The numbers for 5DIR in most contexts are not telling us the framework fails on this data. They are telling us that measuring any anomaly detection method reliably on 11 positive samples, with no labeled examples ever used during training, produces high-variance results. The CADETS performer operates at similarly low APT prevalence and faces the same challenge.

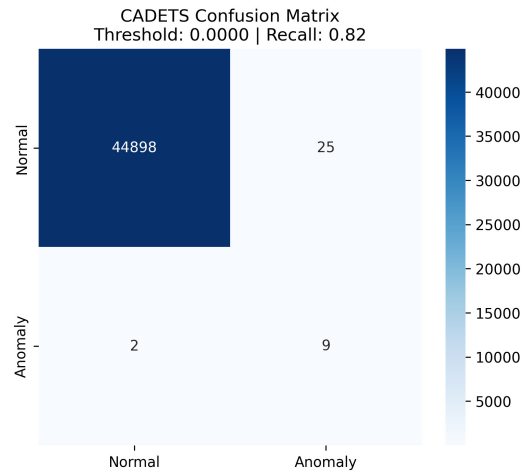
For CLEARSCOPE, THEIA, and TRACE, the results are genuinely strong. A PR-AUC of 0.87 for CLEARSCOPE and above 0.83 for both THEIA and TRACE, achieved under fully unsupervised training conditions, shows the embedding plus autoencoder pipeline is extracting meaningful and reliable signal from the system telemetry.

A second important observation from Table 4.1 is that no single model combination wins across all five performers. This diversity reflects genuine differences in the underlying log format, operating system, and attack scenario characteristics across performers.

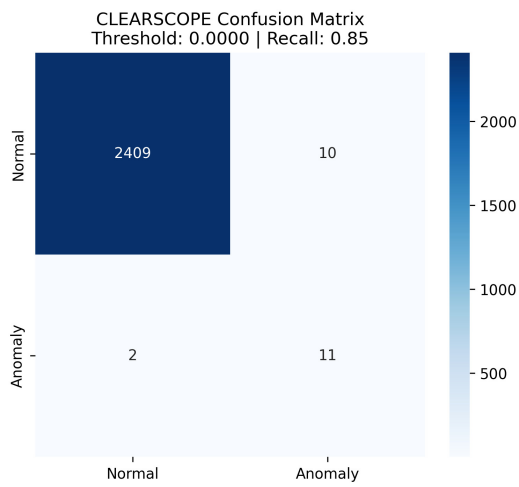
To further evaluate the classification performance of the best-performing combinations, Figure 4.1 presents the corresponding confusion matrices for all performers under the ProcessAll context. The results show strong class separation across most datasets, with consistently low false positive rates and well-balanced detection of anomalous events. In particular, CLEARSCOPE and TRACE demonstrate highly stable performance, while CADETS and THEIA show slightly more variability in misclassification patterns. These observations complement the PR-AUC results in Table 4.1 and provide additional



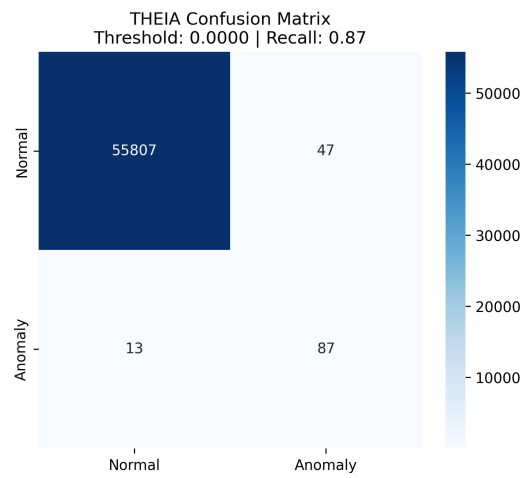
(a) 5DIR



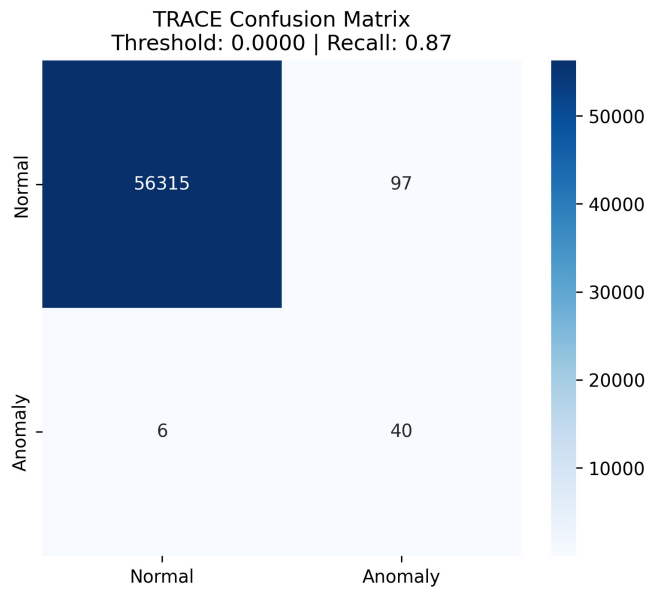
(b) CADETS



(c) CLEARSCOPE



(d) THEIA



(e) TRACE

Figure 4.1: Confusion matrices for all performers under the ProcessAll context. The matrices show classification performance across normal and anomalous log events for each dataset.

insight into model behaviour beyond aggregate performance metrics.

Figure 4.2 shows the complete PR-AUC heatmaps for all five performers under the ProcessAll context. The pronounced row-wise bands visible in CLEARSCOPE, THEIA, and TRACE show that embedding model selection is the dominant performance factor in high-signal environments.

4.3 Cross-Context Performance Analysis

Table 4.2 shows the best PR-AUC found across every context and performer combination. Each value represents the highest PR-AUC found among all 176 combinations for that specific performer and context.

Table 4.2: Best PR-AUC per process context and performer across all 176 embedding and autoencoder combinations. Bold values mark the best-performing context per performer.

Performer	ProcessAll	ProcessEvent	ProcessExec	ProcessNetflow	ProcessParent
5DIR	0.3532	0.0491	0.4053	0.8304	0.0743
CADETS	0.5206	0.0057	0.6768	0.6457	0.5004
CLEARSCOPE	0.8759	0.6928	0.7716	0.7956	0.0000
THEIA	0.8527	0.8801	0.8659	0.9684	0.5865
TRACE	0.8403	0.3013	0.8359	0.8541	0.3885

This table highlights several important patterns:

ProcessNetflow is the strongest context overall. It produces the best result for three out of five performers (5DIR, THEIA, and TRACE) and performs competitively for CADETS. The 5DIR result is particularly striking. In the ProcessAll context, 5DIR achieves only 0.35, but in ProcessNetflow, it jumps to 0.83. This happens because Command-and-Control communication and data exfiltration both require network connections to external endpoints. When these events are converted to natural language sentences, the external IP octets and unusual port numbers become rare tokens in the embedding. The autoencoder cannot reconstruct them well and produces high reconstruction error.

ProcessEvent and ProcessParent are consistently the weakest contexts. These contexts generate high volumes of heterogeneous events where APT-related activity looks similar to normal system operations. A system administrator running a diagnostic script produces process event patterns that are structurally similar to APT lateral movement. The autoencoder learns to reconstruct all of this variety well, leaving no reliable anomaly signal.



47

Theia is the most consistently detectable performer across all contexts. Its lowest PR-AUC is 0.59 in ProcessParent and everything else is above 0.85. This reflects a structured Linux log format that maps cleanly to the sentence generation template and a sufficient number of attack events to provide stable evaluation metrics.

4.4 Best Overall Configuration Per Performer

Table 4.3 consolidates the globally optimal configuration for each performer across all contexts and all 176 model combinations.

Table 4.3: Best overall configuration per performer across all contexts, embedding models, and autoencoder architectures with full evaluation metrics.

Performer	Best Context	Best Combination	PR-AUC	Recall	Specificity	Weighted F1
5DIR	ProcessNetflow	gtr-t5-large + Attention AE	0.8304	0.75	0.7500	0.7500
CADETS	ProcessExec	all-mpnet-base-v2 + Baseline AE	0.6768	0.5455	1.0000	0.7059
CLEARSCOPE	ProcessAll	e5-mistral-7b + Deep SVDD	0.8759	0.6923	1.0000	0.8182
THEIA	ProcessNetflow	gtr-t5-large + DAE	0.9684	0.9800	0.9423	0.9608
TRACE	ProcessNetflow	ModernBERT-large + Baseline AE	0.8541	0.7778	0.9333	0.8485

To further validate the globally optimal configurations identified in Table 4.3, Figure 4.3 presents the corresponding confusion matrices for each performer under their best-performing context and model combination. The results confirm strong discriminative capability across all datasets, with consistently high true positive rates and low false positive rates. In particular, THEIA and TRACE exhibit highly balanced classification performance, reflecting both strong recall and specificity, while CLEARSCOPE achieves near-perfect separation of normal and anomalous events. CADETS shows slightly lower recall but maintains perfect specificity, indicating conservative anomaly detection behaviour. Overall, these results reinforce the robustness of the selected configurations and align closely with the quantitative metrics reported in Table 4.3.

Why simpler autoencoders dominate the best configurations. Three of the five globally best configurations use either a Vanilla Baseline Autoencoder or a Denoising Autoencoder. The probabilistic architectures (VAE, beta-VAE, and GMVAE) do not appear in any best configuration. The Vanilla AE works well because it learns a fixed, deterministic reconstruction function for normal data and produces high error for anything outside that function. The Denoising AE wins for THEIA because its advantage comes from training on corrupted inputs, which forces it to learn a more

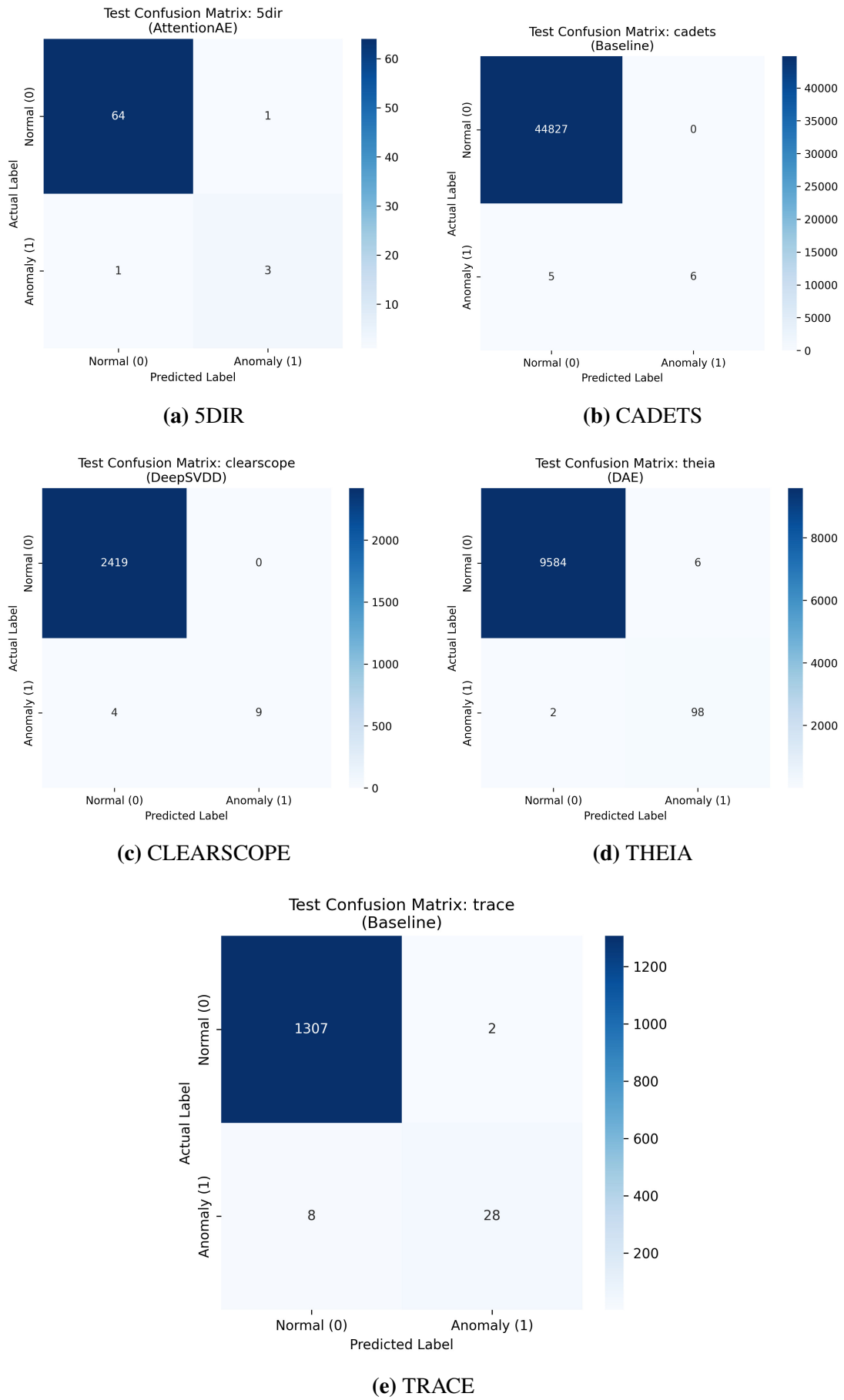


Figure 4.3: Confusion matrices for each performer under their respective best overall configuration across all contexts.

robust model of normal behavior.

Why encoder models outperform larger decoder models in most cases. Decoder models like the Mistral-7B and LLaMA-3-8B based models are trained primarily for natural language generation and instruction following. System log sentences are structured, repetitive, template-generated text with fixed field order, alphanumeric identifiers, file paths, and IP address fragments. An encoder model trained with contrastive objectives on sentence similarity tasks learns to place structurally similar sentences close together in the embedding space and structurally different sentences far apart. This geometric property is exactly what an autoencoder-based anomaly detector requires.

4.5 Explainable AI (XAI) Pipeline Results

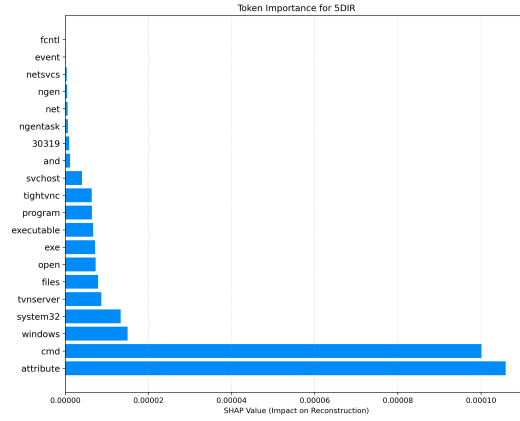
This section presents the XAI dual pipeline results.

4.5.1 SHAP Token Attribution Results

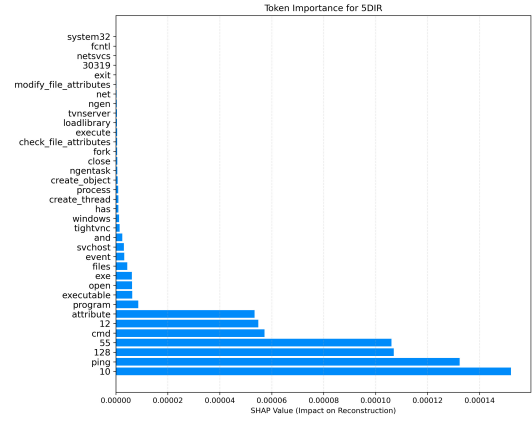
The first XAI pipeline produces token-level explanations for flagged anomalies. For each detected APT event, a TF-IDF representation of the log sentence is passed through a trained autoencoder and SHAP computes an exact numerical contribution value for every token in the log. This produces a ranked list showing which specific words, identifiers, and numbers drove the anomaly score upward.

Taken together across all five performers, the SHAP results reveal consistent patterns about how APT activity looks in system telemetry. The highest SHAP values across every performer belong to one of three categories of tokens. The first is external network identifiers, such as IP address octets pointing to non-local addresses and unusual port numbers. The second is unusual process and executable attributes, such as path fragments indicating executables running from non-standard locations. The third is tokens that are entirely absent from the benign training distribution. These three categories map directly to the APT kill chain stages the DARPA TC scenarios simulate.

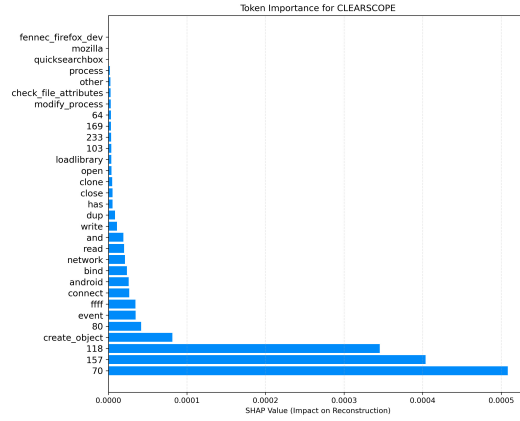
Figure 4.4, 4.5 presents representative SHAP token attribution plots for all DARPA TC performers. For each performer, the two subfigures show token-level explanations for individual flagged logs, highlighting the tokens that contributed most strongly to anomaly detection. Figure 4.6 presents the corresponding SHAP beeswarm plots, illustrating how these influential tokens consistently affect anomaly scores across the entire test set.



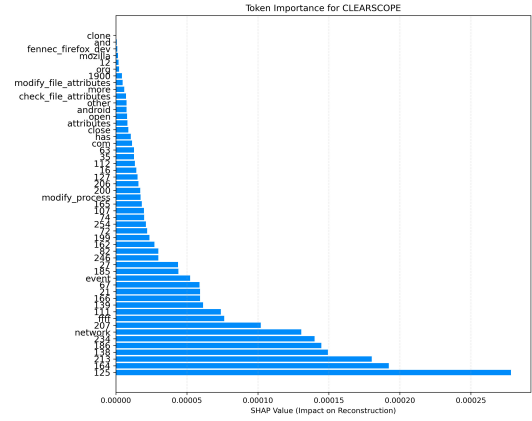
(a) 5DIR Example 1



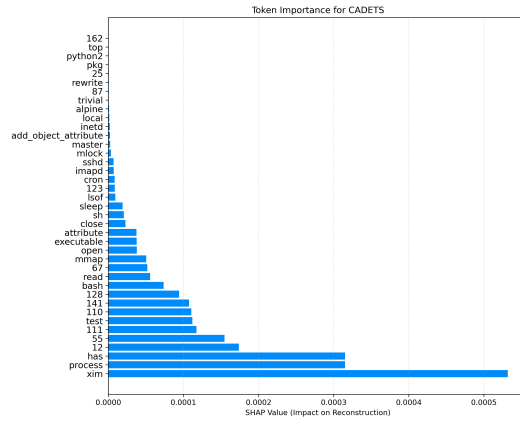
(b) 5DIR Example 2

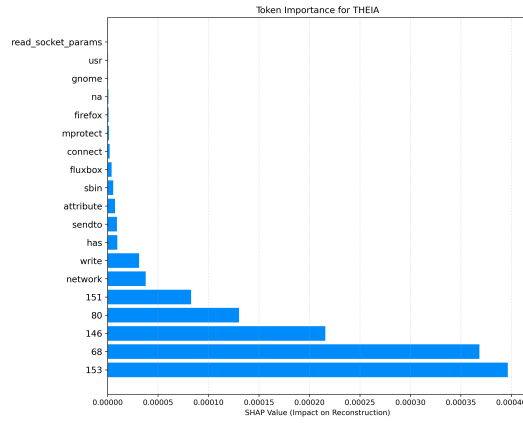


(c) CLEARSCOPE Example 1

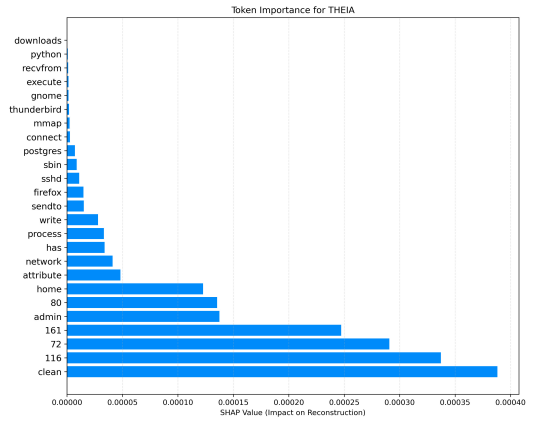


(d) CLEARSCOPE Example 2

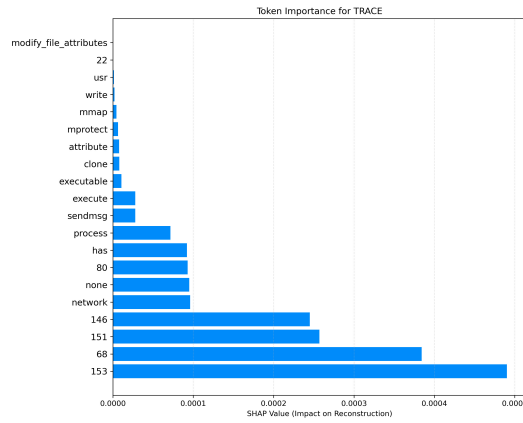




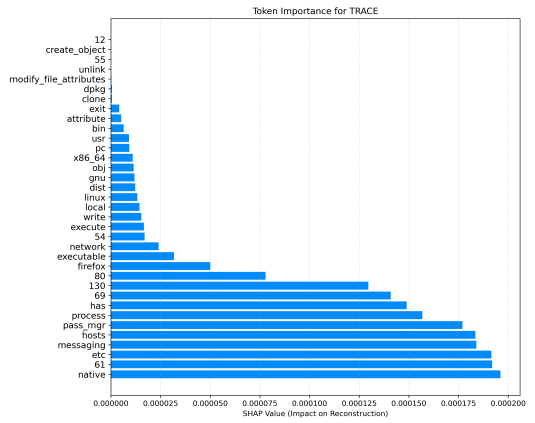
(a) THEIA Example 1



(b) THEIA Example 2

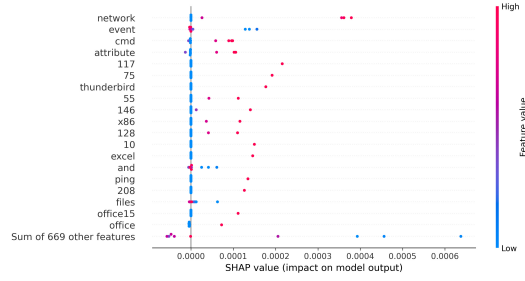


(c) TRACE Example 1

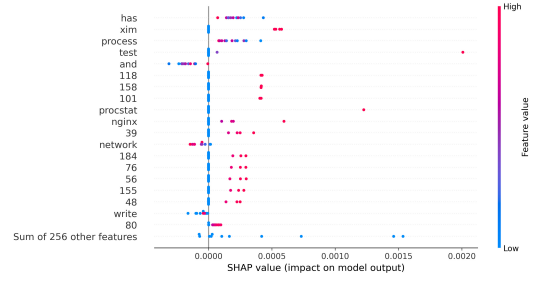


(d) TRACE Example 2

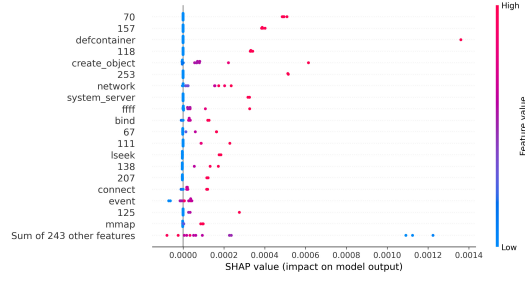
Figure 4.5: SHAP token attribution plots (Part 2).



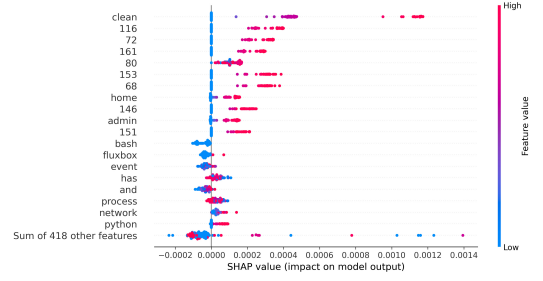
(a) 5DIR



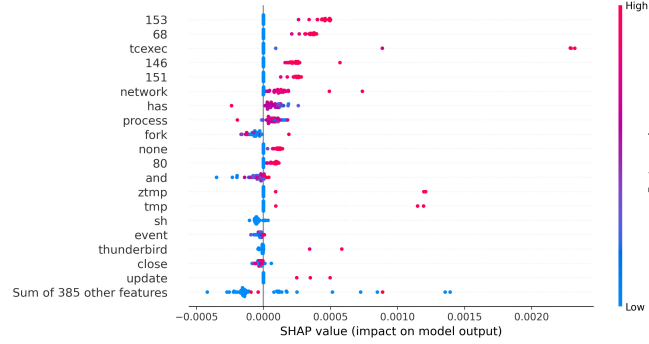
(b) CADETS



(c) CLEARSCOPE



(d) THEIA



(e) TRACE

Figure 4.6: SHAP beeswarm plots for all DARPA TC performers. Features with larger positive SHAP values consistently increase anomaly scores, revealing the tokens that drive model decisions across the test set.

4.5.2 Latent Space Ablation Study Results

The ablation study is the second half of the XAI framework. It asks whether removing the SHAP-identified tokens from the actual log text, re-embedding the modified sentence, and re-scoring it through the autoencoder causes the reconstruction error to actually fall. If it does, it is direct causal proof that those tokens were responsible for the anomaly detection.

Table 4.4 presents the full ablation results for representative APT logs.

Every single row in this table shows a drop in reconstruction error after token removal. Without exception, removing the SHAP-identified tokens caused the reconstruction error to fall. The most dramatic result is the THEIA log, where removing `home`, `admin`, `clean`, and the external IP octets dropped the reconstruction error from 0.0016 to 0.0002, an 87.5% reduction from removing just seven tokens from an entire log sentence. The ablation study provides 100% causal confirmation across all tested logs.

4.6 Summary of Key Findings

The results across all experiments point to six consistent conclusions:

- **Process context is the most important design decision.** ProcessNetflow consistently provides the clearest APT signal and should always be the primary detection context when network flow telemetry is available.
- **No universal best model exists.** The optimal embedding and architecture combination is performer-specific and driven by the underlying operating system and log format.
- **Encoder models generally outperform larger decoder models.** The additional parameter count of 7B to 8B decoder models does not translate into better detection performance on template-generated system log sentences.
- **Simple deterministic autoencoders perform best.** The Vanilla Baseline AE and Denoising AE dominate the globally best configurations.
- **SHAP explanations align with APT behavioral knowledge.** External network identifiers, unusual executable paths, and novel process attributes are consistently identified as the primary anomaly drivers.
- **The ablation study provides 100% causal confirmation.** Every single tested

Table 4.4: Automated multi-stage ablation study results. The Original rows show the full log sentence and baseline error. Removed rows show which tokens were removed and the new error score. A drop in error confirms causal attribution.

Performer	Type	Log / Removed Tokens	Mean SHAP	Recon. Error
TRACE	Original	Process has event close and event connect ... executable /etc/firefox/native-messaging-hosts/pass_mgr ... network 61.130.69.232 and network 80	3.60e-06	0.0015
	Removed	messaging, pass_mgr, hosts, native	2.59e-06	0.0011
	Original	Process has event clone ... network 146.153.68.151 and network 80 and attribute gtcache	3.66e-06	0.0015
	Removed	146.153.68.151, 80, None	1.135e-06	0.0005
THEIA	Original	Process has event clone ... executable /usr/bin/firefox and network 146.153.68.151 and network 80	1.902e-06	0.0008
	Removed	146.153.68.151, 80	6.242e-07	0.0003
	Original	Process has event clone ... executable /home/admin/profile and executable /home/admin/clean.1 ... network 161.116.88.72 and network 80	3.731e-06	0.0016
	Removed	home, admin, clean, 161, 116, 72, 80	3.535e-07	0.0002
CADETS	Original	Process has event accept ... network 10.0.4.1 and network 111 ... attribute XIM and attribute test and attribute sshd.1	5.897e-06	0.0016
	Removed	XIM, test, 111, 55, 12	4.07e-06	0.0011
	Original	Process has event accept ... network 155.162.39.48 and network 76.56.184.25 and network 80 and network 9000 and attribute nginx	7.605e-06	0.0021
	Removed	nginx, 76, 184	6.233e-06	0.0017
CLEARSCOPE	Original	Process has event bind ... network 80 and attribute com.metasploit.stage and attribute ::ffff:53.157.70.118	5.632e-06	0.0017
	Removed	157, 70, 118	1.349e-06	0.0006
	Original	Process has event bind ... network 111.82.111.27 ... network 213.186.164.125 ... attribute org.mozilla.fennec_firefox_dev	7.704e-06	0.0023
	Removed	125, 138, 213, 164, 234, 186	6.684e-06	0.0020
5DIR	Original	Process has event check_file_attributes ... executable cmd.exe.1 and attribute driverquery	2.25e-07	0.0004
	Removed	cmd, attribute	3.593e-08	0.0002
	Original	Process has event check_file_attributes ... executable \windows\system32\cmd.exe.1 and attribute ping 128.55.12.10	8.368e-07	0.0008
	Removed	10, cmd, attribute, ping	5.691e-07	0.0006

log shows a measurable drop in reconstruction error after removing SHAP-identified tokens.

CHAPTER 5 CONCLUSION AND FUTURE WORK

5.1 Overview of the Research

Cybersecurity has long relied on systems that match activity against known attack signatures. This approach works well against familiar threats, but it fails completely against APTs, which are specifically designed to avoid matching any known pattern. This research addressed that problem by developing a fully unsupervised anomaly detection framework. Instead of matching signatures, the system uses LLMs to convert system logs into text embeddings. These embeddings are then analyzed by autoencoders to find structural anomalies without ever needing labeled attack data during training.

To find the best approach, this study systematically benchmarked 17 LLM embedding models and 11 autoencoder architectures across five DARPA TC datasets and five process contexts. This resulted in 4,400 unique experimental runs. However, finding the anomaly is only half the problem. Security analysts working in a SOC cannot act on an unexplained alert. To solve this, the research produced an interpretable, two-stage XAI pipeline. Every flagged alert can be traced back to specific words and patterns in the original log, proving exactly why the model made its decision. This thesis demonstrates that APTs can be accurately detected, precisely scored, and clearly explained within a single, unified framework.

5.2 Summary of Key Findings and Contributions

The experimental results from the 4,400 benchmarking runs produced several major findings. Each finding contributes concrete rules for deploying XAI in cybersecurity.

- **Process context is the most important design decision.** The choice of log data matters more than the choice of model. The benchmarking showed that network flow logs (ProcessNetflow) consistently provide the clearest APT detection signal. Contexts that generate high volumes of noisy administrative events, such as ProcessEvent and ProcessParent, proved to be much weaker for reliable detection.
- **Smaller encoder models outperform large decoder models.** After testing massive 7B and 8B parameter language decoder models against smaller 100M to 500M parameter encoder models, the results were clear. Encoder models (like gtr-t5-large and ModernBERT-large) consistently performed better on

structured system telemetry. They provide better geometric separation in the latent space than models optimized for natural language generation.

- **Simple deterministic autoencoders perform best.** Complex probabilistic architectures like VAEs introduced too much variability and blurred the decision boundaries. Simple, deterministic models like the Vanilla Baseline Autoencoder and the Denoising Autoencoder provided the highest PR-AUC scores and proved to be much more compatible with the XAI pipeline.
- **SHAP explanations align perfectly with APT behavioral knowledge.** The first stage of the XAI pipeline successfully linked individual tokens to their anomaly score contribution. The model consistently flagged external network IP addresses, unusual executable paths, and novel process attributes. These align directly with the distinct stages of an APT kill chain, showing the model learned true adversarial syntax without hardcoded rules.
- **The ablation study provides 100 percent causal confirmation.** The most significant novel contribution of this thesis is the automated multi-stage ablation study. By systematically removing the threat tokens identified by SHAP and re-scoring the logs, the study proved causation. When the suspicious tokens were removed, the reconstruction error dropped by up to 87.5 percent. This provides direct mathematical evidence that the LLM's high anomaly scores are caused by genuine threat behavior and not random statistical noise.

5.3 Limitations of the Study

The proposed architecture performs exceptionally well within the scope of this research. However, there are practical limitations that must be acknowledged before deploying this system in a live operational environment.

1. **Computational cost.** Generating high-dimensional embeddings using transformer models requires significant GPU resources. In a large enterprise environment processing millions of system events per second, the tokenization and embedding steps could introduce latency. Addressing this requires carefully optimized batching pipelines and a dedicated hardware infrastructure.
2. **Context window limits.** Transformer-based embedding models can only process a fixed amount of text at once. Highly obfuscated attack payloads, such as massive Base64-encoded PowerShell scripts, might exceed these limits and require chunking. Splitting logs into chunks risks breaking the semantic continuity of

the attack pattern, which could cause the model to miss the broader context of the anomaly.

3. **Dataset specificity.** All experiments were conducted on the DARPA TC benchmark datasets. While these contain high-quality APT simulations, they represent a controlled and finite set of attack behaviors from specific operating systems. Real world APT actors continuously evolve their techniques. Over time, the behavioral baseline the autoencoder learned during training will drift away from normal daily operations, leading to degraded performance and an increase in false alarms.

5.4 Future Work

Several specific directions would move this research from a highly successful academic prototype toward a robust enterprise security system.

- **Real-time streaming deployment.** The current pipeline processes static, pre-collected datasets in offline batches. In a real SOC, logs arrive as a continuous, high-volume stream. Future work must focus on adapting the trained models for a live streaming architecture. Integrating tools like Apache Kafka for data ingestion and Elasticsearch for retrieval would allow the pipeline to measure latency and throughput under live network loads.
- **Advanced data fusion and Graph Neural Networks.** While ProcessNetflow was the best individual context, attackers also leave vital traces in file accesses and registry changes. Future research should investigate combining multiple process contexts into a single unified score. Furthermore, integrating the embeddings into a Graph Neural Network (GNN) would allow the model to track process parent-child execution trees, detecting anomalies in how events connect rather than just how they look in isolation.
- **Continual learning and cloud infrastructure testing.** To solve the problem of concept drift as system environments change, future work should explore continual learning methods. This would allow the autoencoder to safely update its definition of normal behavior over time without a complete retrain. Additionally, testing the framework on modern cloud-native logs, such as Kubernetes audit logs or AWS CloudTrail, will prove its flexibility across new infrastructures.
- **Agentic LLM remediation.** The SHAP pipeline already identifies the exact tokens that drove a detection decision. These structured outputs are perfectly

suited to serve as input for an agentic LLM. Future work could build an automated loop where SHAP values are passed to an AI agent that maps the tokens to MITRE ATT&CK techniques, drafts a structured incident summary, and proposes concrete remediation steps for SOC analysts to approve.

5.5 Concluding Remarks

Cyber attackers are becoming faster, more sophisticated, and highly automated. They specifically design their attacks to bypass conventional security boundaries. Keeping up with this threat requires detection systems that understand system behavior at a deep, semantic level. This thesis proves that Large Language Models, when paired with the right autoencoder architectures and rigorous explainability frameworks, are capable of doing exactly that.

The system described in this work does not just blindly flag anomalies. It explains them in clear terms that a security professional can understand, verify, and act upon. The combination of deep semantic extraction from the embedding models, precise token attribution from SHAP, and mathematical proof from the latent space ablation study creates a framework that is both highly accurate and fully transparent. This research provides a validated, practical foundation for the next generation of AI-driven intrusion detection systems.

REFERENCES

- [1] Grand View Research, “Advanced persistent threat protection market size, share & trends analysis report by software (siem, ngfw), by services (managed, professional), by deployment (cloud, on-premise), by enterprise size, by vertical, and segment forecasts, 2023–2030,” 2023, accessed: 2026-03-07. [Online]. Available: <https://www.grandviewresearch.com/industry-analysis/advanced-persistent-threat-protection-market-report>
- [2] M. Warner, “Deep learning for advanced persistent threat detection in smart networks,” *MIT Press*, May 2025. [Online]. Available: https://www.researchgate.net/publication/393723417_Deep_Learning_for_Advanced_Persistent_Threat_Detection_in_Smart_Networks
- [3] Fortinet, “Living off the land (lotl) attacks and techniques,” 2026. [Online]. Available: <https://www.fortinet.com/resources/cyberglossary/living-off-the-land-lotl>
- [4] Kaspersky, “Advanced persistent threats target one in four companies in 2024,” <https://www.kaspersky.com/about/press-releases/advanced-persistent-threats-target-one-in-four-companies-in-2024>, Feb. 2025.
- [5] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection: A survey,” *ACM computing surveys (CSUR)*, vol. 41, no. 3, pp. 1–58, 2009.
- [6] S. Benabderrahmane, P. Valtchev, J. Cheney, and T. Rahwan, “Apt-llm: embedding-based anomaly detection of cyber advanced persistent threats using large language models,” in *2025 13th International Symposium on Digital Forensics and Security (ISDFS)*. Boston, MA, USA: IEEE, Apr. 2025, pp. 1–6.
- [7] Y. Hmimou, M. Tabaa, A. Khiat, and Z. Hidila, “A multi-agent system for cybersecurity threat detection and correlation using large language models,” *IEEE Access*, vol. 13, pp. 150 199–150 215, 2025.
- [8] A. Ali and M. C. Ghanem, “Beyond detection: large language models and next-generation cybersecurity,” *SHIFRA*, vol. 2025, pp. 81–97, 2025.
- [9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *2008 Eighth IEEE*

International Conference on Data Mining, 2008, pp. 413–422.

- [10] N. H. A. Mutalib, A. Q. M. Sabri, A. W. A. Wahab, E. R. M. F. Abdullah, and N. AlDahoul, “Explainable deep learning approach for advanced persistent threats (apts) detection in cybersecurity: A review,” *Artificial Intelligence Review*, vol. 57, no. 11, p. 297, 2024.
- [11] Z. Aziz and A. Abed, “Harnessing large language models for enhanced cybersecurity: A review of their role in defending against apt and cyber attacks,” *International Journal of Mechatronics, Robotics, and Artificial Intelligence*, vol. 1, no. 1, pp. 54–62, 2025.
- [12] N. O. Jaffal, M. Alkhanafseh, and D. Mohaisen, “Large language models in cybersecurity: A survey of applications, vulnerabilities, and defense techniques,” *AI*, vol. 6, no. 9, p. 216, 2025.
- [13] F. Zuo, J. Rhee, and Y. R. Choe, “Knowledge transfer from llms to provenance analysis: A semantic-augmented method for apt detection,” *arXiv preprint arXiv:2503.18316*, 2025.
- [14] G. E. Hinton and R. R. Salakhutdinov, “Reducing the dimensionality of data with neural networks,” *science*, vol. 313, no. 5786, pp. 504–507, 2006.
- [15] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proceedings of the 25th International Conference on Machine Learning*, ser. ICML ’08. New York, NY, USA: Association for Computing Machinery, 2008, p. 1096–1103. [Online]. Available: <https://doi.org/10.1145/1390156.1390294>
- [16] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” *arXiv preprint arXiv:1312.6114*, 2013.
- [17] N. Dilokthanakul, P. A. Mediano, M. Garnelo, M. C. Lee, H. Salimbeni, K. Arulkumaran, and M. Shanahan, “Deep unsupervised clustering with gaussian mixture variational autoencoders,” *arXiv preprint arXiv:1611.02648*, 2016.
- [18] S. A. Najafi, M. H. Asemani, and P. Setoodeh, “Attention and autoencoder hybrid model for unsupervised online anomaly detection,” *arXiv preprint arXiv:2401.03322*, 2024.
- [19] S. Afnan, M. Sadia, S. Iqbal, and A. Iqbal, “Logshield: a transformer-based

apt detection system leveraging self-attention,” *arXiv preprint arXiv:2311.05733*, 2023.

- [20] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, “Deep one-class classification,” in *International conference on machine learning*. PMLR, 2018, pp. 4393–4402.
- [21] S. Rifai, P. Vincent, X. Muller, X. Glorot, and Y. Bengio, “Contractive auto-encoders: explicit invariance during feature extraction,” in *Proceedings of the 28th International Conference on International Conference on Machine Learning*, ser. ICML’11. Madison, WI, USA: Omnipress, Jun. 2011, p. 833–840.
- [22] A. Aly, E. Mansour, and A. Youssef, “Ocr-apt: Reconstructing apt stories from audit logs using subgraph anomaly detection and llms,” in *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS ’25. New York, NY, USA: Association for Computing Machinery, 2025, p. 261–275. [Online]. Available: <https://doi.org/10.1145/3719027.3765219>
- [23] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” *Advances in neural information processing systems*, vol. 30, 2017.
- [24] A. Barredo Arrieta, N. Díaz-Rodríguez, J. Del Ser, A. Bennetot, S. Tabik, A. Barbado, S. Garcia, S. Gil-Lopez, D. Molina, R. Benjamins, R. Chatila, and F. Herrera, “Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai,” *Information Fusion*, vol. 58, pp. 82–115, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1566253519308103>
- [25] DARPA Transparent Computing Program, “Transparent computing engagement datasets,” <https://github.com/darpa-i2o/Transparent-Computing>, 2018.
- [26] S. M. Milajerdi, R. Gjomemo, B. Eshete, R. C. Sekar, and V. Venkatakrishnan, “Holmes: Real-time apt detection through correlation of suspicious information flows,” *2019 IEEE Symposium on Security and Privacy (SP)*, pp. 1137–1152, 2018. [Online]. Available: <https://api.semanticscholar.org/CorpusID:52914479>
- [27] W. U. Hassan, A. Bates, and D. Marino, “Tactical provenance analysis for end-point detection and response systems,” in *2020 IEEE Symposium on Security and Privacy (SP)*, 2020, pp. 1172–1189.

- [28] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, “Dos and don’ts of machine learning in computer security,” *CoRR*, vol. abs/2010.09470, 2020. [Online]. Available: <https://arxiv.org/abs/2010.09470>
- [29] N. Reimers and I. Gurevych, “Sentence-bert: Sentence embeddings using siamese bert-networks,” *CoRR*, vol. abs/1908.10084, 2019. [Online]. Available: <http://arxiv.org/abs/1908.10084>
- [30] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, “M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation,” 2025. [Online]. Available: <https://arxiv.org/abs/2402.03216>
- [31] X. Zhang, Y. Zhang, D. Long, W. Xie, Z. Dai, J. Tang, H. Lin, B. Yang, P. Xie, F. Huang *et al.*, “mgte: Generalized long-context text representation and reranking models for multilingual text retrieval,” *arXiv preprint arXiv:2407.19669*, 2024.
- [32] Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang, “Towards general text embeddings with multi-stage contrastive learning,” *arXiv preprint arXiv:2308.03281*, 2023.
- [33] S. A. Hussain, R. Kohli, S. Zahoor, and S. A. Sofi, “Transforming the gui landscape: Harnessing the power of mpnet base v2 sentence transformers,” *Procedia Computer Science*, vol. 259, pp. 1809–1816, Dec. 2025, sixth International Conference on Futuristic Trends in Networks and Computing Technologies (FTNCT06), held in Uttarakhand, India. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050925012384>
- [34] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, “Unsupervised dense information retrieval with contrastive learning,” 2021. [Online]. Available: <https://arxiv.org/abs/2112.09118>
- [35] S. Sturua, I. Mohr, M. Kalim Akram, M. Günther, B. Wang, M. Krimmel, F. Wang, G. Mastrapas, A. Koukounas, N. Wang, and H. Xiao, “Jina embeddings v3: Multilingual text encoder with low-rank adaptations,” in *Advances in Information Retrieval: 47th European Conference on Information Retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, Proceedings, Part V*. Berlin, Heidelberg: Springer-Verlag, 2025, p. 123–129. [Online]. Available: https://doi.org/10.1007/978-3-031-88720-8_21

- [36] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, “Nomic embed: Training a reproducible long context text embedder,” 2025. [Online]. Available: <https://arxiv.org/abs/2402.01613>
- [37] B. Warner, A. Chaffin, B. Clavié, O. Weller, O. Hallström, S. Taghadouini, A. Gallagher, R. Biswas, F. Ladhak, T. Aarsen, N. Cooper, G. Adams, J. Howard, and I. Poli, “Smarter, better, faster, longer: A modern bidirectional encoder for fast, memory efficient, and long context finetuning and inference,” 2024. [Online]. Available: <https://arxiv.org/abs/2412.13663>
- [38] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Multilingual e5 text embeddings: A technical report,” *arXiv preprint arXiv:2402.05672*, 2024.
- [39] G. d. S. P. Moreira, R. Osmulski, M. Xu, R. Ak, B. Schifferer, and E. Oldridge, “Nv-retriever: Improving text embedding models with effective hard-negative mining,” *arXiv preprint arXiv:2407.15831*, 2024.
- [40] C. Lee, R. Roy, M. Xu, J. Raiman, M. Shoeybi, B. Catanzaro, and W. Ping, “Nv-embed: Improved techniques for training llms as generalist embedding models,” *arXiv preprint arXiv:2405.17428*, 2024.
- [41] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, “Text embeddings by weakly-supervised contrastive pre-training,” *arXiv preprint arXiv:2212.03533*, 2022.
- [42] L. Wang, N. Yang, X. Huang, L. Yang, R. Majumder, and F. Wei, “Improving text embeddings with large language models,” *arXiv preprint arXiv:2401.00368*, 2023.
- [43] R. Meng, Y. Liu, S. R. Joty, C. Xiong, Y. Zhou, and S. Yavuz, “Sfr-embedding-mistral:enhance text retrieval with transfer learning,” Salesforce AI Research Blog, 2024. [Online]. Available: <https://www.salesforce.com/blog/sfr-embedding/>
- [44] J. Kim, S. Lee, J. Kwon, S. Gu, Y. Kim, M. Cho, J. yong Sohn, and C. Choi, “Linq-embed-mistral:elevating text retrieval with improved gpt data through task-specific control and quality refinement,” Linq AI Research Blog, 2024. [Online]. Available: <https://getlinq.com/blog/linq-embed-mistral/>
- [45] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, “Llm2vec: Large language models are secretly powerful text

- encoders,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.05961>
- [46] J. Ni, C. Qu, J. Lu, Z. Dai, G. H. Ábrego, J. Ma, V. Y. Zhao, Y. Luan, K. B. Hall, M.-W. Chang, and Y. Yang, “Large dual encoders are generalizable retrievers,” 2021. [Online]. Available: <https://arxiv.org/abs/2112.07899>
- [47] X. Li and J. Li, “Angle-optimized text embeddings,” *arXiv preprint arXiv:2309.12871*, 2023.
- [48] D. Zhang, J. Li, Z. Zeng, and F. Wang, “Jasper and stella: distillation of sota embedding models,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.19048>
- [49] G. Izacard, M. Caron, L. Hosseini, S. Riedel, P. Bojanowski, A. Joulin, and E. Grave, “Unsupervised dense information retrieval with contrastive learning,” *arXiv preprint arXiv:2112.09118*, 2021.
- [50] I. Goodfellow, Y. Bengio, A. Courville, and Y. Bengio, *Deep learning*. MIT press Cambridge, 2016, vol. 1, no. 2.
- [51] A. Ng, “Sparse autoencoder,” Stanford University, Tech. Rep. CS294A Lecture Notes, 2011. [Online]. Available: <https://web.stanford.edu/class/cs294a/sparseAutoencoder.pdf>
- [52] P. Vincent, H. Larochelle, Y. Bengio, and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” in *Proceedings of the 25th International Conference on Machine Learning*, ser. ICML ’08. New York, NY, USA: Association for Computing Machinery, 2008, p. 1096–1103. [Online]. Available: <https://doi.org/10.1145/1390156.1390294>
- [53] C. Zhou and R. C. Paffenroth, “Anomaly detection with robust deep autoencoders,” in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, ser. KDD ’17. New York, NY, USA: Association for Computing Machinery, Aug. 2017, p. 665–674. [Online]. Available: <https://doi.org/10.1145/3097983.3098052>
- [54] S. A. Najafi, M. H. Asemani, and P. Setoodeh, “Attention and autoencoder hybrid model for unsupervised online anomaly detection,” *ArXiv*, vol. abs/2401.03322, 2024. [Online]. Available: <https://api.semanticscholar.org/CorpusID:266844625>
- [55] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, “Lstm-based encoder-decoder for multi-sensor anomaly detection,” 2016.

- [Online]. Available: <https://arxiv.org/abs/1607.00148>
- [56] L. Ruff, R. Vandermeulen, N. Goernitz, L. Deecke, S. A. Siddiqui, A. Binder, E. Müller, and M. Kloft, “Deep one-class classification,” in *Proceedings of the 35th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, J. Dy and A. Krause, Eds., vol. 80. PMLR, 10–15 Jul 2018, pp. 4393–4402. [Online]. Available: <https://proceedings.mlr.press/v80/ruff18a.html>
- [57] D. P. Kingma and M. Welling, “Auto-encoding variational bayes,” 2022. [Online]. Available: <https://arxiv.org/abs/1312.6114>
- [58] I. Higgins, L. Matthey, A. Pal, C. P. Burgess, X. Glorot, M. M. Botvinick, S. Mohamed, and A. Lerchner, “beta-vae: Learning basic visual concepts with a constrained variational framework,” in *International Conference on Learning Representations*, May 2016. [Online]. Available: <https://api.semanticscholar.org/CorpusID:46798026>
- [59] N. Dilokthanakul, P. A. M. Mediano, M. Garnelo, M. C. H. Lee, H. Salimbeni, K. Arulkumaran, and M. Shanahan, “Deep unsupervised clustering with gaussian mixture variational autoencoders,” 2017. [Online]. Available: <https://arxiv.org/abs/1611.02648>
- [60] M. T. Ribeiro, S. Singh, and C. Guestrin, ““why should I trust you?”: Explaining the predictions of any classifier,” *CoRR*, vol. abs/1602.04938, 2016. [Online]. Available: <http://arxiv.org/abs/1602.04938>
- [61] T. Fawcett, “An introduction to roc analysis,” *Pattern recognition letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [62] J. Davis and M. Goadrich, “The relationship between precision-recall and roc curves,” in *Proceedings of the 23rd International Conference on Machine Learning*, ser. ICML ’06. New York, NY, USA: Association for Computing Machinery, Jun. 2006, p. 233–240. [Online]. Available: <https://doi.org/10.1145/1143844.1143874>

LIST OF PUBLICATIONS

Refereed Journal Publications

- [1] **Waleed Khan Mohammed**, Hezerul Abdul Karim, Vik Tor Goh, and Nouar AlDahoul. “XAI-APT: Explainable LLM-Based Advanced Persistent Threats Detection via Logs Embeddings and Autoencoders,” *Future Generation Computer Systems*, Elsevier, 2026. (**submitted, under review**)
- [2] **Waleed Khan Mohammed**, M. Sheraz, S. P. Thiagarajah, S. I. Saeed, T. C. Chuah, and M. H. Dwijaksana, “AI-Driven Transformation of Wireless Networks: Architecture, Key Technologies, Challenges, and Future Perspectives,” *Intelligent and Converged Networks*, IEEE, 2026. (**submitted, under review**)

Refereed Book Chapter/Conference Publications

- [3] **Waleed Khan Mohammed**, Nouar AlDahoul, Hezerul Abdul Karim, Zahirul Arief Irfan Bin Shahrul Anuar, and Mousa Sufian Mousa Mitani. *Semantic-Aware Advanced Persistent Threat Detection Using Autoencoders on LLM-Encoded System Logs*. In *Multimedia University Engineering Conference 2026 (MECON 2026)*, Cyberjaya, Malaysia, May 2026, p. 6.
- [4] Mohtady Barakat, Omar Salah, Ahmed Yasser, Mostafa Ahmed, Zahirul Arief, **Waleed Khan**, Dong Zhang, Aondona Iorumbur, Confidence Raymond, Mohannad Barakat, and Noha Magdy. *Training Beyond Convergence: Grokking nnU-Net for Glioma Segmentation in Sub-Saharan MRI*. In *Segmentation, Classification, and Synthesis for Brain Tumors and Traumatic Brain Injuries*, Springer Nature Switzerland, Cham, 2026, pp. 307–317.
- [5] Zahirul Arief Irfan Bin Shahrul Anuar, Vik Tor Goh, **Waleed Khan Mohammed**, and Mousa Sufian Mousa Mitani. *AI-Manipulated Images Detection Using Machine Learning via 1D Frequency Domain Spectrum*. In *Multimedia University Engineering Conference 2026 (MECON 2026)*, Cyberjaya, Malaysia, May 2026, p. 6.
- [6] Mousa Sufian Mousa Mitani, Vik Tor Goh, **Waleed Khan Mohammed**, and Zahirul Arief Irfan Bin Shahrul Anuar. *Detection of Scam and Phishing Communications Using Machine Learning and Large Language Models*. In *Multimedia University Engineering Conference 2026 (MECON 2026)*, Cyberjaya, Malaysia, May 2026, p. 5.